

Aplicação de Lista de Compras com Preços e Alertas

SHOPISEL



Autores: Martim Monteiro — N.º 51619
Alexandre Tavares — N.º 51271
Pedro Gomes — N.º 51423

Orientador: Doutor Fernando Miguel Gamboa de Carvalho, IPL/ISEL
João Pereira

Agradecimentos

Gostaríamos de expressar o nosso sincero agradecimento ao Doutor Fernando Miguel Gamboa de Carvalho pela sua disponibilidade constante e pela rapidez nas respostas ao longo do desenvolvimento deste projeto, contributos esses que se revelaram fundamentais para a sua concretização.

Agradecemos igualmente ao João Pereira, ao Pedro Tainha e ao Breno Nico pela idealização do projeto, bem como por todas as explicações, esclarecimentos e orientações fornecidas, que foram essenciais para a definição da estrutura, abordagem e formas de pensar adotadas ao longo deste trabalho.

A todos, o nosso muito obrigado pelo apoio e colaboração.

Declaração de integridade

Declaramos que este trabalho de projeto é o resultado da nossa investigação pessoal e independente. O seu conteúdo é original e todas as fontes listadas nas referências bibliográficas foram consultadas e estão devidamente mencionadas no texto. Mais declaramos que todas as referências científicas e técnicas relevantes para o desenvolvimento do trabalho estão devidamente citadas e constam das referências bibliográficas.

Os autores

Lisboa, 20 de Julho de 2026

Declaração de Utilização de Inteligência Artificial

A utilização de inteligência artificial (IA) constituiu um eixo transversal do presente trabalho, alinhado com uma das propostas orientadoras do projeto: explorar formas de acelerar o desenvolvimento em colaboração com ferramentas de IA, mantendo simultaneamente a qualidade, a integridade e a responsabilidade técnica sobre o código e o texto produzidos.

Ao longo de todas as fases do projeto — desde a conceção inicial e definição de arquitetura até à implementação, testes, documentação técnica, configuração de pipelines de integração contínua e redação deste relatório — recorreu-se a assistentes baseados em modelos de linguagem de grande dimensão (*Large Language Models*). As ferramentas utilizadas incluem o GitHub Copilot, o ChatGPT, o Codex e o Claude.

Estas ferramentas foram empregues, nomeadamente, para sugestão e geração de código, apoio à escrita de testes automatizados, explicação de erros e *debugging*, elaboração e reformulação de texto técnico, tradução de conteúdos, revisão de lógica de negócio e aceleração de tarefas repetitivas ou de maior volume documental. Em todos os casos, os resultados obtidos foram revistos, validados e adaptados por nós (autores), e assim, assumimos a responsabilidade final pelo conteúdo integrado no sistema e descrito neste documento.

A abordagem adotada não substituiu o raciocínio crítico nem as decisões de engenharia: tratou-se de um trabalho em conjunto com a IA, em que os autores mantiveram o controlo sobre as escolhas arquiteturais, a conformidade com os requisitos funcionais, a execução de testes e a verificação do comportamento do sistema em ambiente real. O objetivo foi aumentar a produtividade sem comprometer a fiabilidade da solução desenvolvida.

Abstract

This project presents the development of a mobile application for managing shopping lists enhanced with price comparison and alert functionalities, supported by a service-oriented backend architecture. The solution is designed to be used in real-world contexts, such as supermarkets, allowing users to efficiently create and manage shopping lists, mark items during purchases, and access updated product prices across different stores and locations.

The system integrates multiple technologies, including a React Native mobile application, a React-based web application for shopping preparation and analysis, and a set of RESTful backend services developed in .NET. Authentication and authorization are handled by Keycloak, while push notifications are delivered through Firebase Cloud Messaging to inform users about price drops.

A central data management component is responsible for handling products, prices, and store information, ensuring consistency and reliability. The architecture follows a modular, service-oriented approach, enabling the integration of additional services and supporting data synchronization requirements.

Additionally, the project explores the concept of White Label development, allowing the customization of applications through dedicated CI/CD pipelines. The use of AI-powered tools, such as GitHub Copilot, is also incorporated to support development and validation processes.

Overall, this work demonstrates how a seemingly simple application can evolve into an advanced functional prototype with an extensible foundation for future features, addressing challenges related to distributed systems, data management, and real-time user interaction.

Resumo

O presente projeto apresenta o desenvolvimento de uma aplicação móvel para gestão de listas de compras, enriquecida com funcionalidades de comparação de preços e definição de alertas, suportada por uma arquitetura de backend orientada a serviços. A solução foi concebida para apoiar o planeamento de compras em contexto de supermercado, permitindo aos utilizadores criar e gerir listas de compras, registar itens durante a compra e consultar preços atualizados de produtos em diferentes lojas e localizações.

O sistema integra diversas tecnologias, incluindo uma aplicação móvel desenvolvida em React Native, uma aplicação web em React para preparação e análise de compras, e um conjunto de serviços backend com APIs REST implementados em .NET. A autenticação e autorização são asseguradas através do Keycloak, enquanto as notificações são geridas com recurso ao Firebase Cloud Messaging, permitindo alertar os utilizadores para descidas de preços.

A solução inclui ainda um mecanismo centralizado de gestão de dados, responsável pela manutenção de informação relativa a produtos, preços e lojas, de modo a preservar a integridade e a coerência da informação entre serviços. A arquitetura adotada segue princípios modulares, com separação clara de responsabilidades entre serviços, o que facilita a evolução do sistema e a integração de novos serviços.

Adicionalmente, o projeto explora o conceito de desenvolvimento White Label, permitindo a customização das aplicações através de pipelines dedicados de CI/CD. Foi também utilizada inteligência artificial, nomeadamente ferramentas baseadas em LLMs como o GitHub Copilot, como apoio ao desenvolvimento e validação do sistema.

Em suma, este trabalho demonstra como uma aplicação com um âmbito funcional inicial delimitado pode evoluir para um protótipo funcional avançado, com uma base extensível para novas funcionalidades, abordando desafios relacionados com sistemas distribuídos, gestão de dados e interação em tempo real com o utilizador.

Índice

Agradecimentos	i
Declaração de integridade	ii
Declaração de Utilização de Inteligência Artificial	iii
Abstract	iv
Resumo	v
Siglas e Acrónimos	xi
1 Introdução	1
1.1 Contexto e Motivação	1
1.2 Abordagem	1
1.3 Objetivos	2
1.4 Requisitos do Sistema	2
1.4.1 Requisitos Funcionais	2
1.4.2 Requisitos Não Funcionais	2
1.5 Estrutura do Relatório	3
2 Estado da Arte	4
2.1 Arquitetura orientada a serviços	4
2.2 Persistência de dados e modelo relacional	5
2.3 Aplicações web modernas	5
2.4 Aplicações móveis multiplataforma	5
2.5 Autenticação e autorização	6
2.6 Recolha automática de dados	6
2.7 Containerização, CI/CD e deployment	7
2.8 Síntese	7
3 Trabalho Relacionado	8
3.1 Aplicações de gestão de listas de compras	8
3.2 Plataformas de comparação de preços	9
3.3 Aplicações próprias de cadeias de supermercados	9

3.4	Análise comparativa e posicionamento do ShopISEL	10
3.5	Síntese	10
4	Trabalho Desenvolvido	12
4.1	Visão Geral da Arquitetura	12
4.1.1	Modelo de Dados	13
4.2	Serviços de Backend	15
4.2.1	List Service	15
4.2.1.1	Tratamento de erros e concorrência otimista	16
4.2.2	Products Service	17
4.2.3	Prices Service	17
4.2.4	Stores Service	18
4.2.5	Account Service	19
4.3	Pipeline de Recolha de Dados	19
4.3.1	Scraper Orchestrator	19
4.3.2	Scraper Continente	20
4.3.3	Scraper Pingo Doce	20
4.3.4	MatchQueue	21
4.4	FCM Notification Worker	21
4.5	Aplicação Web	22
4.5.1	Integração com Concorrência Otimista	23
4.5.2	Arquitetura white-label dinâmica	23
4.6	Aplicação Móvel	24
4.6.1	Leitura de código de barras	24
4.6.2	Localização e preços próximos	24
4.7	Autenticação e Autorização	25
4.8	Gateway e Infraestrutura de Deployment	26
5	Processo de Desenvolvimento e Integração Contínua	28
5.1	Organização do Projeto em Repositórios	28
5.2	Controlo de Versões e Estratégia de Branches	29
5.3	Pipelines de Integração e Entrega Contínua	29
5.4	Reusable Workflows e Orquestração de Scrapers	30
5.5	Ferramentas de Suporte ao Desenvolvimento	30
5.6	Gestão de Configuração e Segredos	31
5.7	Desenvolvimento White Label	31
6	Testes e Avaliação	33
6.1	Estratégia de Testes	33
6.2	Infraestrutura de Testes	34
6.2.1	Base de dados em memória com SQLite	34
6.2.2	Handler de autenticação de teste	35

6.3	Casos de Teste do List Service	35
6.3.1	Fluxo CRUD completo	35
6.3.2	Isolamento por proprietário	35
6.3.3	Filtragem no GET /lists	36
6.3.4	Conflito de concorrência	36
6.4	Execução dos Testes nas Pipelines de CI/CD	36
6.5	Avaliação Funcional	37
7	Trabalho Futuro	38
7.1	Evolução Funcional	38
7.2	Recolha de Dados e Inteligência Artificial	39
7.3	Resiliência e Infraestrutura	39
7.4	Distribuição da Aplicação Móvel	40
8	Conclusão	41
8.1	Síntese do Trabalho Desenvolvido	41
8.2	Desafios Encontrados	42
8.3	Considerações Finais	43
	Bibliografia	45

Lista de Figuras

4.1	Visão geral da arquitetura do sistema ShopISEL	12
4.2	Pipeline de recolha de dados e entrega de notificações	13
4.3	Modelo de dados simplificado do sistema ShopISEL. A relação <code>parentId</code> auto-referencial da entidade <i>Category</i> está omitida por clareza visual.	14
4.4	Fluxo de autenticação com Keycloak	26
5.1	Fluxo de Continuous Integration/Continuous Delivery (CI/CD) para os serviços de backend	29

Lista de Tabelas

3.1	Comparação entre soluções existentes e o ShopISEL	10
4.1	Entidades de dados por serviço, complementando o modelo simplificado da Figura 4.3.	14

Siglas e Acrónimos

API	Application Programming Interface
CI/CD	Continuous Integration/Continuous Delivery
HTTP	HyperText Transfer Protocol
IA	Inteligência Artificial
JSON	JavaScript Object Notation
JWT	JSON Web Token
OIDC	OpenID Connect
REST	Representational State Transfer
URL	Uniform Resource Locator
FCM	Firebase Cloud Messaging
OCR	Optical Character Recognition
SQL	Structured Query Language
SMTP	Simple Mail Transfer Protocol

Capítulo 1

Introdução

O ShopISEL é uma plataforma digital de apoio ao planeamento de compras, que centraliza informação de produtos e preços de múltiplas insígnias de retalho. A plataforma permite criar e gerir listas de compras, comparar preços entre lojas, receber alertas de descida de preço e identificar as opções mais vantajosas com base na localização do utilizador. A solução é disponibilizada através de uma aplicação móvel, orientada ao contexto real de compra, e de uma aplicação web, para preparação e análise em computador.

1.1 Contexto e Motivação

O processo de planeamento de compras é frequentemente moroso: os preços variam entre lojas e promoções, obrigando o utilizador a consultar múltiplas fontes para tomar uma decisão informada. As soluções existentes tendem a focar-se num único domínio — gestão de listas ou comparação de preços — sem os integrar numa experiência coesa. Acresce que a maioria das plataformas de comparação de preços se destina ao comércio eletrónico, não cobrindo adequadamente as lojas físicas de supermercado.

O ShopISEL procura colmatar esta lacuna, combinando gestão de listas, comparação de preços entre insígnias, alertas automáticos e georreferenciação de lojas numa única plataforma integrada.

1.2 Abordagem

A solução segue uma arquitetura orientada a serviços, com o backend dividido em serviços independentes por domínio (listas, produtos, preços, lojas, contas de utilizador), expostos através de Application Programming Interface (API)s REST. A recolha automática de dados é assegurada por scrapers dedicados por insígnia, sem dependência de APIs proprietárias. A autenticação é centralizada no Keycloak, e as notificações push são entregues via Firebase Cloud Messaging. Todo o sistema é containerizado com Docker e suportado por pipelines de CI/CD com GitHub Actions.

1.3 Objetivos

Os objetivos principais do projeto ShopISEL são:

1. Desenvolver uma plataforma integrada para criação e gestão de listas de compras, consulta de preços em múltiplas lojas e alertas de descida de preço;
2. Implementar recolha automática de dados de produtos e preços de insígnias de retalho portuguesas, sem dependência de APIs proprietárias;
3. Disponibilizar a plataforma em aplicação móvel (Android e iOS) e aplicação web;
4. Integrar georreferenciação de lojas para apresentar as opções mais próximas e vantajosas;
5. Garantir autenticação segura baseada em padrões abertos (OpenID Connect (OIDC) / OAuth 2.0);
6. Assegurar qualidade e entrega contínua através de pipelines de CI/CD automatizadas.

1.4 Requisitos do Sistema

1.4.1 Requisitos Funcionais

1. O sistema deve permitir criar, consultar, atualizar e eliminar listas de compras pessoais;
2. O sistema deve apresentar preços atualizados de produtos em múltiplas lojas e insígnias;
3. O sistema deve notificar o utilizador quando o preço de um produto favorito descer;
4. O sistema deve suportar a identificação de produtos por leitura de código de barras;
5. O sistema deve apresentar lojas ordenadas por proximidade geográfica;
6. O sistema deve autenticar e autorizar utilizadores, garantindo isolamento de dados entre contas de utilizador;
7. O sistema deve recolher automaticamente dados de produtos e preços sem dependência de APIs proprietárias.

1.4.2 Requisitos Não Funcionais

1. **Segurança:** autenticação com padrões abertos (OIDC / OAuth 2.0) e dados de sessão cifrados;
2. **Escalabilidade:** adição de novos serviços e insígnias sem impacto nos componentes existentes;
3. **Disponibilidade:** deployment automático em plataformas de cloud, tirando partido dos mecanismos de disponibilidade da infraestrutura;
4. **Manutenibilidade:** ciclo de build, teste e deployment independente por serviço;

1.5 Estrutura do Relatório

- **Capítulo 2 — Estado da Arte:** tecnologias e ferramentas adotadas e respectiva justificação;
- **Capítulo 3 — Trabalho Relacionado:** análise comparativa de soluções existentes;
- **Capítulo 4 — Trabalho Desenvolvido:** arquitetura, componentes e implementação da plataforma;
- **Capítulo 5 — Processo de Desenvolvimento e Integração Contínua:** organização dos repositórios e pipelines de CI/CD;
- **Capítulo 6 — Testes e Avaliação:** estratégia de testes e avaliação funcional do sistema;
- **Capítulo 7 — Trabalho Futuro:** temas e melhorias identificados mas não desenvolvidos por restrições de tempo;
- **Capítulo 8 — Conclusão:** síntese do trabalho e desafios encontrados.

Capítulo 2

Estado da Arte

Este capítulo apresenta o estado da arte das tecnologias, metodologias e ferramentas mais relevantes para o desenvolvimento da plataforma ShopISEL. A análise é orientada pelas necessidades concretas do projeto: disponibilizar uma solução composta por aplicação web, aplicação móvel, serviços de backend, persistência relacional, autenticação centralizada, recolha automática de dados de produtos e mecanismos de entrega contínua.

2.1 Arquitetura orientada a serviços

Uma tendência consolidada no desenvolvimento de plataformas distribuídas é a separação do sistema em serviços especializados. Esta abordagem permite dividir responsabilidades por domínio funcional, reduzir o acoplamento entre componentes e facilitar a evolução independente de cada parte do sistema. No projeto ShopISEL, esta separação encontra-se refletida em serviços distintos para listas de compras, produtos, lojas e preços.

A utilização de APIs Representational State Transfer (REST) continua a ser uma escolha frequente em sistemas web e móveis, devido à sua simplicidade, interoperabilidade e boa integração com clientes baseados em HyperText Transfer Protocol (HTTP). Estudos recentes sobre frameworks para serviços web destacam que a escolha da tecnologia de backend deve considerar desempenho, escalabilidade, produtividade e maturidade do ecossistema [28]. No caso do ShopISEL, estes critérios justificam a adoção de serviços em ASP.NET Core, com endpoints organizados por domínio e contratos explícitos para a comunicação com as aplicações cliente.

O projeto utiliza Minimal APIs em ASP.NET Core, uma abordagem recomendada para a criação de APIs HTTP com menor quantidade de código estrutural e boa adequação a serviços pequenos ou focados num domínio específico [18]. Esta opção é coerente com a divisão do repositório, onde cada serviço encapsula o seu modelo de dados, endpoints, lógica de negócio e configuração de execução.

2.2 Persistência de dados e modelo relacional

A gestão de produtos, lojas, listas e preços exige consistência entre entidades relacionadas. Por esse motivo, as bases de dados relacionais continuam a ser uma solução adequada para sistemas onde existem relações fortes, restrições de integridade e necessidade de consultas estruturadas. PostgreSQL é uma das soluções relacionais mais utilizadas em aplicações modernas, sendo mantida com documentação oficial e suporte para versões atuais [26].

No ShopISEL, os serviços de backend recorrem a Entity Framework Core com o provider Npgsql para PostgreSQL. Esta combinação permite modelar entidades em C# (C Sharp), aplicar migrações e interagir com a base de dados através de uma camada de acesso a dados integrada no ecossistema .NET [22]. A utilização de migrações é particularmente importante porque o sistema evolui por serviços: cada domínio pode alterar o seu esquema de dados sem exigir uma reorganização global da aplicação.

Esta opção também facilita o suporte a cenários relevantes para o projeto, como a associação de itens a listas de compras, a categorização de produtos, o registo de lojas e a manutenção de informação de preços recolhida de fontes externas. A persistência relacional oferece, assim, uma base estável para relacionar dados operacionais com funcionalidades de comparação e consulta.

2.3 Aplicações web modernas

As aplicações web modernas privilegiam interfaces reativas, comunicação assíncrona com APIs e componentes reutilizáveis. React consolidou-se como uma biblioteca amplamente adotada para construção de interfaces declarativas, tirando partido de componentes e hooks para gerir estado, efeitos e composição de comportamentos [17]. Vite, por sua vez, é frequentemente utilizado em projetos frontend por simplificar a configuração e acelerar o ciclo de desenvolvimento.

No ShopISEL, a aplicação web é construída com React, TypeScript e Vite. Esta combinação permite desenvolver uma interface orientada a componentes para consultar produtos, gerir listas, visualizar alertas e interagir com os serviços de backend. O uso de TypeScript acrescenta verificação estática ao código cliente, reduzindo erros em estruturas de dados trocadas com as APIs. A presença de bibliotecas de componentes e utilitários de interface, como Material UI, Radix UI e Tailwind, permite acelerar a construção de ecrãs consistentes sem abandonar a personalização visual da plataforma.

2.4 Aplicações móveis multiplataforma

No contexto de compras presenciais, a aplicação móvel é essencial porque acompanha o utilizador no local onde a decisão de compra acontece. Funcionalidades como consulta rápida de listas, leitura de códigos de barras, comparação de preços e receção de alertas dependem

de uma experiência adaptada ao dispositivo móvel.

React Native permite desenvolver aplicações móveis com uma base tecnológica próxima do ecossistema React, enquanto Expo fornece ferramentas, módulos nativos e serviços que simplificam a criação de aplicações para Android, iOS e web a partir de um projeto JavaScript/TypeScript [5]. No repositório do ShopISEL, a aplicação móvel usa Expo, Expo Router, React Native, NativeWind e módulos como Expo Camera e Expo Secure Store. Esta escolha é adequada ao projeto porque permite integrar navegação por separadores, autenticação, armazenamento seguro e leitura por câmara, mantendo uma base de desenvolvimento coerente com a aplicação web.

2.5 Autenticação e autorização

Sistemas que manipulam listas pessoais e preferências de utilizador necessitam de autenticação fiável e de uma separação clara entre identidade e lógica aplicacional. Keycloak é uma solução de gestão de identidade e acesso que suporta protocolos como OpenID Connect e OAuth 2.0, permitindo proteger aplicações e serviços através de tokens e endpoints padronizados [12].

No ShopISEL, Keycloak é utilizado como componente central de autenticação. O serviço de listas valida tokens JSON Web Token (JWT) emitidos por Keycloak, verificando emissor, validade e partes autorizadas antes de permitir o acesso a recursos protegidos. Esta abordagem evita que cada serviço implemente autenticação própria e permite que web, mobile e backend partilhem um modelo comum de identidade.

2.6 Recolha automática de dados

Uma plataforma de comparação de preços depende da qualidade e atualização dos dados. Quando não existe uma API pública uniforme para todas as lojas, a recolha automática por scraping torna-se uma alternativa prática, embora exija cuidados com estrutura das páginas, normalização de dados e robustez perante alterações externas.

O projeto inclui scrapers específicos para Continente e Pingo Doce. Embora sejam componentes independentes, no sentido em que cada loja online tem a sua própria estrutura de DOM, navegação e regras de extração, ambos perseguem o mesmo objetivo funcional: recolher automaticamente produtos por categoria e extrair informação como nome, preço, promoção, imagem e disponibilidade. Esta separação permite adaptar a lógica de scraping às particularidades de cada insígnia sem afetar as restantes. Os resultados são persistidos em MongoDB, utilizando a variável de ambiente `SCRAPER_MONGO_CONNECTION` para configurar a ligação. No caso do Pingo Doce, a utilização de Playwright é adequada a páginas dinâmicas, uma vez que esta ferramenta permite automatizar browsers e interagir com conteúdo renderizado no cliente. A existência de ficheiros JavaScript Object Notation (JSON) de saída também facilita a validação dos dados extraídos e a análise posterior por categoria.

2.7 Containerização, CI/CD e deployment

A entrega de aplicações compostas por vários serviços exige mecanismos que garantam reprodutibilidade e automatização. Docker permite empacotar aplicações e dependências em containers, tornando mais previsível a execução em ambientes distintos [4]. Esta característica é especialmente relevante no ShopISEL, onde coexistem serviços .NET, frontend web, gateway Nginx e componentes de suporte.

O repositório inclui Dockerfiles e workflows de GitHub Actions para automatizar validação, construção e publicação de imagens. GitHub Actions suporta a automatização de workflows diretamente no repositório, incluindo tarefas de integração e entrega contínua [7]. A configuração de deployment usa Fly.io, com serviços baseados em imagens e um Nginx configurado como ponto de entrada para encaminhar pedidos para o frontend e os serviços de backend.

2.8 Síntese

O estado da arte analisado mostra que a arquitetura escolhida para o ShopISEL está alinhada com práticas atuais de desenvolvimento de aplicações distribuídas. A separação em serviços facilita a evolução por domínio, PostgreSQL e Entity Framework Core oferecem uma base consistente para persistência, React e Expo permitem cobrir web e mobile com tecnologias próximas, Keycloak centraliza autenticação e Docker com GitHub Actions apoia a entrega contínua.

Estas escolhas tecnológicas estão alinhadas com os objetivos do projeto: centralizar informação de produtos e preços, suportar a gestão de listas de compras, permitir a comparação entre lojas e preparar a plataforma para evolução futura com novas fontes de dados, novas funcionalidades de alerta e melhorias na experiência do utilizador.

Capítulo 3

Trabalho Relacionado

As soluções digitais de apoio a compras evoluíram de simples listas locais para plataformas capazes de combinar catálogos de produtos, histórico de preços, alertas e informação contextual. No mercado de supermercados esta evolução é particularmente relevante: o preço de um mesmo produto pode variar consoante a loja, a campanha promocional e a localização geográfica. O ShopISEL posiciona-se neste espaço, relacionando produtos, categorias, lojas e preços de forma integrada, com uma lista de compras inteligente como ponto de entrada.

Este capítulo analisa soluções existentes no domínio das aplicações de listas de compras e comparação de preços, com o objetivo de identificar as limitações das abordagens atuais e justificar as opções tomadas no desenvolvimento do ShopISEL. A análise divide-se em três categorias principais: aplicações de gestão de listas de compras, plataformas de comparação de preços e aplicações próprias de cadeias de supermercados.

3.1 Aplicações de gestão de listas de compras

O mercado de aplicações de listas de compras é relativamente maduro, com diversas soluções estabelecidas que oferecem funcionalidades base de criação e partilha de listas.

O **Bring!** é uma das aplicações mais populares na Europa para gestão de listas de compras. Permite criar listas colaborativas, partilhá-las com membros da família ou amigos em tempo real, e sugere produtos com base no histórico de compras. A interface é intuitiva e suporta múltiplos idiomas, incluindo o português. No entanto, o Bring! foca-se exclusivamente na gestão de listas, não oferecendo comparação de preços, alertas de variação de preço ou georreferenciação de lojas [3].

O **AnyList** destaca-se pela integração com receitas culinárias, permitindo ao utilizador adicionar automaticamente ingredientes de uma receita à lista de compras. Oferece partilha em tempo real entre utilizadores e suporta organização por categorias. À semelhança do Bring!, não inclui funcionalidades de comparação de preços nem notificações sobre alterações de preço [1].

O **OurGroceries** é uma solução orientada para o contexto familiar, com sincronização automática entre dispositivos. Destaca-se pela simplicidade e rapidez de utilização, mas man-

têm o mesmo padrão de ausência de informação de preços, scrapers de dados ou integração com sistemas de retalho [10].

Em comum, estas aplicações partilham a limitação de se focarem exclusivamente na gestão de listas sem qualquer mecanismo de integração com dados de preços reais, tornando-se instrumentos de registo e não de apoio à decisão de compra.

3.2 Plataformas de comparação de preços

Existe um conjunto de soluções orientadas especificamente à comparação de preços entre lojas, algumas com foco no mercado português.

O **Kuanto Kusta** é uma plataforma portuguesa de comparação de preços que agrega produtos de diferentes lojas e permite consultar valores e variações ao longo do tempo. No contexto deste trabalho, é um exemplo relevante porque mostra o tipo de serviço que o Shopl-SEL pretende complementar com gestão de listas, alertas e contexto geográfico. Ainda assim, a plataforma continua focada na consulta de preços e não integra funcionalidades de gestão de listas de compras, leitura de código de barras ou notificações de descida de preço [13].

O **Pricena**, popular em alguns mercados, agrega preços de produtos de diferentes revendedores online, permitindo a comparação e a configuração de alertas de preço. Porém, o seu foco está no comércio eletrónico, não cobrindo lojas físicas de supermercados, e não oferece funcionalidades de gestão de listas [24].

Plataformas internacionais como o **Flipp** (disponível nos EUA e Canadá) oferecem uma abordagem mais abrangente, agregando folhetos digitais de supermercados e permitindo criar listas de compras associadas a promoções. No entanto, esta solução não está disponível no mercado português e depende da adesão voluntária das cadeias de retalho para fornecer dados [6].

A limitação transversal a estas plataformas é a falta de integração com ferramentas de gestão de listas e a ausência de funcionalidades de georreferenciação que permitam ao utilizador identificar a loja mais vantajosa na sua proximidade.

3.3 Aplicações próprias de cadeias de supermercados

As principais cadeias de supermercados portuguesas disponibilizam as suas próprias aplicações móveis, que oferecem funcionalidades como listas de compras digitais, cartão de fidelidade e promoções personalizadas.

A aplicação do **Continente** permite ao utilizador criar listas de compras, consultar promoções, aceder ao cartão de fidelidade e efetuar encomendas para entrega ou levantamento. A aplicação é bem desenvolvida e oferece uma experiência integrada, mas está confinada ao ecossistema do Continente, não permitindo comparação com outras insígnias [25].

De forma análoga, a aplicação do **Pingo Doce** oferece funcionalidades de lista de compras, promoções e cartão de fidelidade, mas igualmente restrita ao universo da sua cadeia [11].

A limitação fundamental destas aplicações é precisamente a sua natureza fechada: cada insígnia apresenta apenas os seus próprios preços e promoções, tornando impossível ao utilizador comparar diretamente preços entre cadeias concorrentes a partir de uma única interface.

3.4 Análise comparativa e posicionamento do ShopISEL

A Tabela 3.1 apresenta uma síntese comparativa entre as soluções analisadas e o ShopISEL, considerando as funcionalidades mais relevantes para o utilizador final.

Tabela 3.1 Comparação entre soluções existentes e o ShopISEL

Funcionalidade	Bring!	Kuanto Kusta	Continente	Flipp	ShopISEL
Gestão de listas	✓	–	✓	✓	✓
Comparação de preços	–	✓	–	✓	✓
Múltiplas insígnias	–	✓	–	✓	✓
Alertas de preço	–	✓	–	–	✓
Leitura de código de barras	–	–	✓	–	✓
Georreferenciação de lojas	–	–	–	–	✓
Aplicação móvel	✓	✓	✓	✓	✓
Aplicação web	–	✓	✓	–	✓
Notificações push	–	–	✓	–	✓

A análise evidencia que nenhuma das soluções existentes combina, numa única plataforma, a gestão de listas de compras com a comparação de preços entre múltiplas insígnias, alertas de variação de preço e georreferenciação de lojas. O ShopISEL posiciona-se precisamente neste espaço, oferecendo uma solução integrada que une estas funcionalidades, disponível tanto em aplicação móvel como em aplicação web.

Adicionalmente, a arquitetura do ShopISEL, orientada a serviços com scrapers independentes por insígnia, permite a adição de novas cadeias de retalho sem impacto na estrutura central do sistema, o que constitui uma vantagem significativa face a soluções fechadas ou dependentes da adesão voluntária dos retalhistas.

3.5 Síntese

O levantamento de soluções existentes demonstra que existe uma lacuna clara no mercado para uma plataforma integrada de gestão de compras que combine listas, comparação de preços entre insígnias, alertas e geolocalização. As soluções atuais focam-se geralmente num

único domínio funcional, obrigando o utilizador a recorrer a múltiplas aplicações para cobrir as suas necessidades. O ShopISEL procura colmatar esta lacuna, oferecendo um protótipo integrado e centrado na experiência de compra do utilizador.

Capítulo 4

Trabalho Desenvolvido

Este capítulo apresenta em detalhe a arquitetura, as tecnologias e a implementação dos componentes que constituem o sistema ShopISEL. A solução é composta por múltiplos serviços de backend, duas aplicações cliente (web e móvel), um conjunto de scrapers de dados, um worker de notificações e a infraestrutura de suporte à autenticação, persistência e deployment.

4.1 Visão Geral da Arquitetura

O ShopISEL segue uma arquitetura orientada a serviços, em que cada domínio funcional é tratado por um serviço autónomo com a sua própria lógica de negócio e ciclo de deployment, partilhando uma única base de dados PostgreSQL. Na generalidade dos casos, cada serviço gere exclusivamente as suas próprias tabelas; o MatchQueue é uma exceção documentada na Secção 4.3.4, por ler e escrever diretamente nas tabelas `products`, `categories`, `prices` e `favorite_products`, geridas em primeira instância pelos serviços de domínio correspondentes. Esta separação permite que a maioria dos serviços evolua de forma independente, reduz o acoplamento entre componentes e facilita a escalabilidade horizontal de cada parte do sistema.

A Figura 4.1 apresenta uma visão geral da arquitetura do sistema.

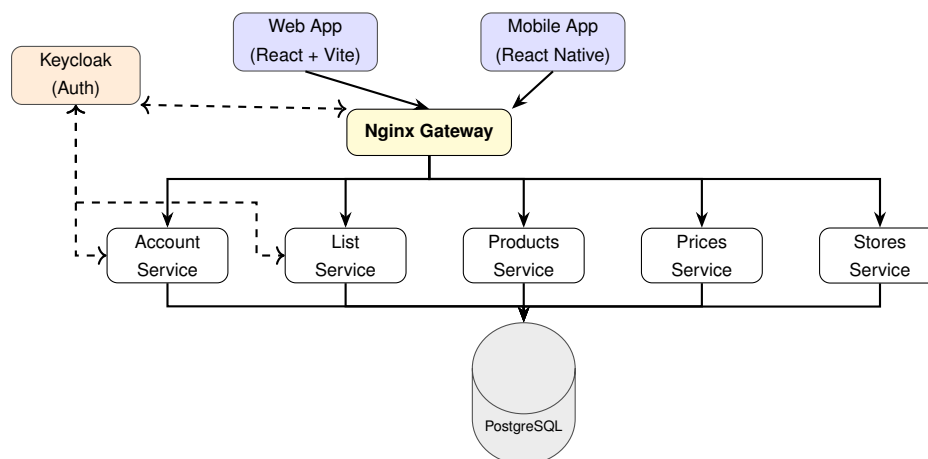


Figura 4.1 Visão geral da arquitetura do sistema ShopISEL

Os pedidos das aplicações cliente chegam ao sistema através de um gateway Nginx, que encaminha cada pedido para o serviço responsável com base no caminho do URL. Os serviços de backend partilham uma única base de dados PostgreSQL e, no caso dos serviços protegidos, validam os tokens JWT emitidos pelo Keycloak.

A recolha de dados de produtos e preços é feita por scrapers independentes para cada insígnia, que persistem os dados em MongoDB como área de *staging* — um espaço de armazenamento intermédio onde os dados brutos, tal como extraídos de cada website, ficam temporariamente disponíveis antes de serem validados e integrados no modelo de dados definitivo, sem que erros ou inconsistências nesta fase inicial afetem diretamente a base de dados relacional em produção. O MatchQueue processa estes dados, normalizando-os e integrando-os na base de dados relacional. Por fim, o FCM Notification Worker lê as notificações pendentes e envia alertas para os dispositivos dos utilizadores através do Firebase Cloud Messaging. A Figura 4.2 apresenta este fluxo de forma separada, para evitar sobreposição com a arquitetura principal.

Por uma questão de foco, a Figura 4.2 representa apenas os componentes de execução e o fluxo de dados entre eles. O mecanismo que desencadeia cada etapa — workflows de GitHub Actions que invocam os scrapers e, de seguida, o MatchQueue e o FCM Notification Worker através de eventos `repository_dispatch` — não é um componente de runtime e não está representado no diagrama; a orquestração completa é descrita na Secção 5.4.

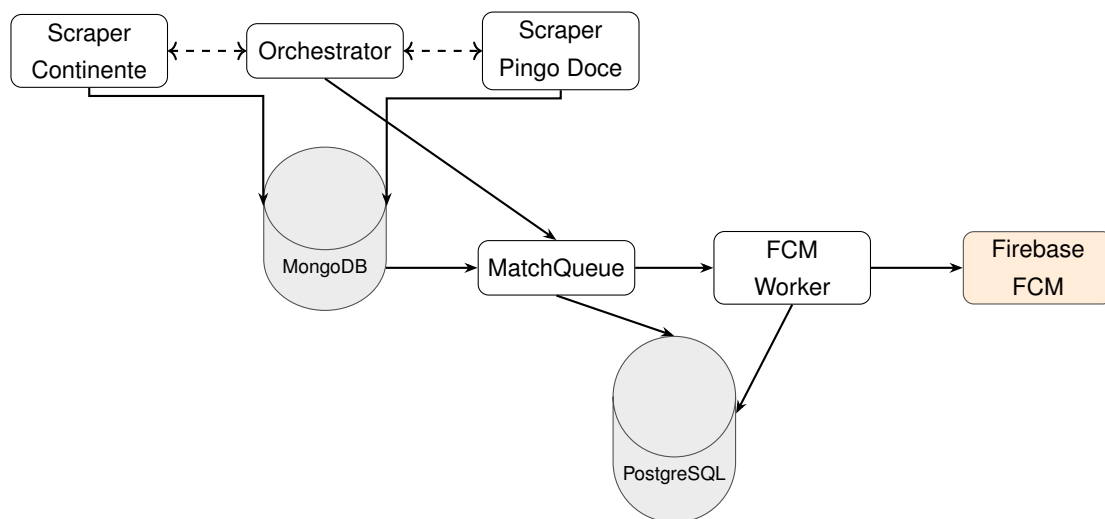


Figura 4.2 Pipeline de recolha de dados e entrega de notificações

4.1.1 Modelo de Dados

A Figura 4.3 apresenta o modelo de dados simplificado do sistema, evidenciando as principais entidades e as relações entre elas. As entidades estão distribuídas pelos diferentes serviços de backend, sendo cada uma gerida pelo serviço responsável pelo respetivo domínio funcional.

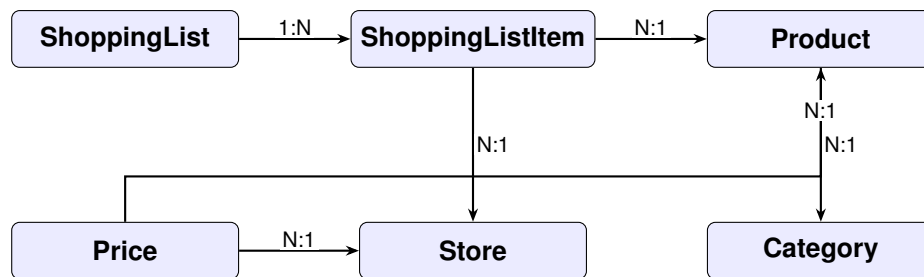


Figura 4.3 Modelo de dados simplificado do sistema ShopISEL. A relação `parentId` auto-referencial da entidade `Category` está omitida por clareza visual.

A Figura 4.3 omite, por clareza visual, um conjunto de entidades de suporte relacionadas com contas de utilizador, notificações e o pipeline de recolha de dados. A Tabela 4.1 completa o modelo, listando todas as entidades persistidas pelos serviços de backend, incluindo as entidades do MatchQueue e os dados brutos armazenados em MongoDB pelos *scrapers*.

Tabela 4.1 Entidades de dados por serviço, complementando o modelo simplificado da Figura 4.3.

Serviço	Entidade	Campos principais
List Service	ShoppingList	Id, OwnerId, Name, CreatedAt, Version
List Service	ShoppingListItem	Id, ListId, ProductId, StoreId, Quantity, Checked
Products Service	Product	Id, Name, Brand, Barcode, Image, CategoryId
Products Service	Category	Id, Name, Image, ParentCategoryId
Prices Service	Price	Id, ProductId, StoreId, PriceText, SaleText, QuantityText, UnitPriceText, SaleDate, UpdatedAt
Stores Service	Store	Id, Name, Brand, Address, City, Latitude, Longitude
Account Service	Account	Id
Account Service	AccountPushToken	Id, AccountId, FcmToken, Platform, DeviceId, IsActive, LastSeenAt, CreatedAt, UpdatedAt
Account Service	FavoriteProduct	AccountId, ProductId
MatchQueue	NotificationEnqueue	Id, AccountId, ProductId, Checked
MongoDB (scrapers)	scrape_runs	_id, worker_run_id, source, status, started_at, finished_at, products_count
MongoDB (scrapers)	scrape_products	_id, run_id, source, product_key, external_name, price_text, sale_text, unit_price_text, category_id, store_id, scraped_at

`NotificationEnqueue` é a única entidade da tabela exclusiva do MatchQueue. As restantes tabelas que o MatchQueue utiliza (`products`, `categories`, `prices`, `favorite_products`) já constam desta tabela associadas aos serviços que as gerem em primeira instância; o MatchQueue acede diretamente às mesmas tabelas físicas através de um modelo Entity Framework Core independente, como descrito na Secção 4.3.4.

4.2 Serviços de Backend

Todos os serviços de backend são desenvolvidos em ASP.NET Core com Minimal APIs, que reduzem a quantidade de código *boilerplate* — código repetitivo e de estrutura, sem lógica de negócio própria, que seria necessário declarar para cada endpoint (por exemplo, classes de *controller* e respectivos atributos de rota) — face à abordagem tradicional baseada em *controllers*, mantendo a integração completa com o ecossistema .NET. Cada serviço dispõe do seu próprio Dockerfile e pipeline de CI/CD, sendo publicado como uma imagem Docker no GitHub Container Registry.

4.2.1 List Service

O List Service é o serviço central do ShopISEL no que diz respeito à interação do utilizador. Gere a criação, consulta, atualização e eliminação de listas de compras, bem como a coleção de itens associada a cada lista.

O modelo de dados do List Service é composto por duas entidades principais:

- **ShoppingListEntity**: representa uma lista de compras, com um identificador único, o identificador do proprietário (`OwnerId`, obtido a partir do claim `sub` do token JWT; neste contexto, um *claim* é uma afirmação codificada no token que transporta atributos do utilizador autenticado), o nome da lista, a data de criação e um token de versão (`Version`) utilizado para controlo de concorrência otimista.
- **ShoppingListItemEntity**: representa um item da lista, com referência ao identificador do produto (`ProductId`), ao identificador da loja (`StoreId`), à quantidade desejada e ao estado de marcação (`Checked`).

Toda a persistência é assegurada pelo Entity Framework Core com o provider Npgsql para PostgreSQL, com migrações aplicadas automaticamente no arranque do serviço.

A autenticação é obrigatória em todos os endpoints do List Service. O serviço valida tokens JWT emitidos pelo Keycloak, verificando o emissor, a validade temporal e o claim `azp` (authorized party), que deve corresponder a um dos clientes registados (aplicação web ou móvel). Desta forma, cada lista de compras é sempre associada ao utilizador autenticado, e as operações de leitura e escrita são automaticamente filtradas por `OwnerId`, garantindo isolamento entre utilizadores.

Os endpoints expostos são:

- GET `/lists` — devolve todas as listas do utilizador autenticado;
- POST `/lists` — cria uma nova lista com a coleção completa de itens fornecida;
- GET `/lists/{listId}` — devolve uma lista específica (apenas se pertencer ao utilizador autenticado);
- PUT `/lists/{listId}` — atualiza o nome e a coleção completa de itens de uma lista, exigindo o header `If-Match` com a versão esperada;
- DELETE `/lists/{listId}` — elimina uma lista, também condicionada ao header `If-Match`.

As respostas de criação e consulta incluem o campo `version` no corpo e o header `ETag`, permitindo às aplicações cliente conhecerem a versão corrente antes de qualquer operação de escrita. Este campo é a base do mecanismo de concorrência otimista adotado no serviço: cada atualização bem-sucedida produz uma nova versão, e qualquer pedido baseado numa versão desatualizada é rejeitado.

O serviço inclui também um endpoint de saúde em `GET /health`, utilizado pelos mecanismos de monitorização da plataforma de deploy.

4.2.1.1 Tratamento de erros e concorrência otimista

O List Service expõe operações de escrita sobre listas que podem ser invocadas em simultâneo a partir de diferentes clientes — por exemplo, quando o mesmo utilizador edita a mesma lista na aplicação web e na aplicação móvel. Sem mecanismos de proteção, a segunda escrita poderia sobrescrever alterações feitas entretanto pelo primeiro cliente, originando perda silenciosa de dados (*lost updates*).

Para resolver este problema, foi adotada uma estratégia de concorrência otimista, seguindo o princípio descrito por King [14]. Cada lista possui um identificador de versão do tipo `Guid`, gerido pela aplicação e persistido na base de dados como token de concorrência do Entity Framework Core. Sempre que uma lista é criada ou atualizada com sucesso, recebe uma nova versão; nas operações de atualização e eliminação, o serviço compara a versão recebida no header `If-Match` com a versão atualmente armazenada. Se coincidirem, a operação prossegue; se outro cliente tiver alterado a lista entretanto, o Entity Framework Core deteta o conflito e a API responde com o código HTTP 409 (*Conflict*).

Esta abordagem foi preferida a alternativas como *rowversion* específico de SQL Server, o token nativo `xmin` do PostgreSQL ou locking pessimista com transações longas. A opção escolhida mantém o comportamento portátil entre PostgreSQL, utilizado em produção, e SQLite, utilizado nos testes de integração, sem bloquear recursos de base de dados enquanto o utilizador interage com a interface.

Paralelamente, o tratamento de erros deixou de estar disperso pelos endpoints e passou a ser centralizado num *middleware* de exceções. Este componente traduz falhas de validação, pedidos malformados, conflitos de concorrência, indisponibilidade temporária da base de dados e erros inesperados em respostas HTTP consistentes, com registo em *log* antes de devolver o resultado ao cliente. As respostas de erro de negócio intencionais — como recurso inexistente (404) ou utilizador não autenticado (401) — continuam a ser tratadas ao nível dos endpoints, mantendo separação entre fluxos esperados da aplicação e falhas de infraestrutura.

O impacto na base de dados materializou-se numa migração que adicionou a coluna `version` à tabela `lists`, inicializando as linhas existentes com um valor por defeito. Novas listas passam a receber automaticamente um identificador de versão no momento da criação.

4.2.2 Products Service

O Products Service gere o catálogo de produtos e categorias da plataforma. Não requer autenticação, sendo acessível tanto pelas aplicações cliente como pelos serviços internos.

O modelo de dados contempla duas entidades:

- **ProductEntity**: representa um produto com identificador, nome, código de barras (*Barcode*), URL de imagem e referência à categoria.
- **CategoryEntity**: representa uma categoria hierárquica, com suporte a subcategorias através de uma relação auto-referencial (*ParentCategoryId*).

A hierarquia de categorias permite organizar os produtos de forma estruturada (e.g., Alimentação → Laticínios → Leite), facilitando a navegação e filtragem na interface do utilizador.

Os endpoints de produto suportam filtragem por identificadores, por categoria e por nome, bem como uma pesquisa de produtos relacionados com base em favoritos do utilizador, com parâmetros de limite e distância máxima:

- GET `/products?ids=...` — pesquisa por identificadores;
- GET `/products?categoryId=...` — filtragem por categoria;
- GET `/products?name=...` — pesquisa por nome;
- GET `/products/related?favoriteIds=...` — produtos relacionados com os favoritos.

O fluxo de correspondência usado no `/products/qrCode` segue duas fases. Primeiro é tentada uma correspondência exata pelo `barcode`, procurando no catálogo um produto com o mesmo código. Se isso falhar, o backend normaliza as palavras-chave recebidas, divide-as em tokens úteis e remove termos comuns que não ajudam na identificação. Depois executa uma filtragem inicial sobre o nome e a marca dos produtos para reduzir o conjunto de candidatos e aplica um *ranking* interno com base em cobertura de palavras-chave, número de tokens coincidentes, semelhança textual, distância de Levenshtein [15] e um pequeno reforço quando a marca também coincide. Se a correspondência for suficientemente forte, o produto é devolvido como *exact*; caso contrário, são devolvidos produtos *related* para apoiar a escolha do utilizador.

O mesmo critério de distância e semelhança é reutilizado na funcionalidade `/products/related`, onde o objetivo deixa de ser validar um `barcode` e passa a ser encontrar produtos próximos dos favoritos do utilizador. Nesse caso, os nomes dos produtos favoritos são normalizados e comparados com candidatos da mesma categoria, usando a distância de Levenshtein para ordenar os resultados por proximidade lexical e devolver uma lista curta de sugestões mais prováveis.

4.2.3 Prices Service

O Prices Service mantém o preço mais recente de cada produto por loja. A entidade **PriceEntity** contém o identificador do produto, o identificador da loja, e os valores tal como extraídos pelos *scrapers* em texto livre: *PriceText* (preço normal), *SaleText* (preço promocional op-

cional), `QuantityText` e `UnitPriceText` (quantidade e preço unitário, quando disponíveis na página de origem), a data da promoção (`SaleDate`) e a data da última atualização. Os preços são guardados como texto porque refletem o formato exato apresentado pela insígnia de origem (por exemplo, “1,99 €”); a conversão para valor numérico, necessária para comparação, ordenação e cálculo de percentagem de desconto, é feita pelas aplicações cliente.

Cada atualização de preço para o mesmo par produto/loja substitui o registo anterior: o `MatchQueue`, ao processar novos dados de *scraping*, localiza o preço existente para esse produto e loja e atualiza-o *in place*. O sistema não preserva, por isso, os valores anteriores de preço — apenas o preço e a eventual promoção correntes são conhecidos em cada momento. A extensão do modelo para preservar histórico de variações está identificada como trabalho futuro (Capítulo 7).

Os endpoints expostos são:

- `GET /prices?productId={id}` — devolve os preços de um produto em todas as lojas disponíveis;
- `GET /prices?productId={id}&storeId={id}` — devolve o preço de um produto numa loja específica;
- `POST /prices` — regista ou atualiza o preço de um produto numa loja (utilizado internamente pelo pipeline de recolha de dados);
- `GET /health` — endpoint de monitorização.

4.2.4 Stores Service

O Stores Service gere o catálogo de lojas do sistema. Cada loja tem um identificador, nome, insígnia (`Brand`), morada, cidade e coordenadas geográficas opcionais (`Latitude/Longitude`), sendo estas últimas usadas para ordenar lojas por proximidade na aplicação cliente.

Na prática, o catálogo distingue dois tipos de registo. Por um lado, existem duas lojas genéricas por insígnia (`store-1`, *Continente Online*, e `store-2`, *Pingo Doce Online*), sem coordenadas, que são as únicas lojas a que os *scrapers* de produtos associam preços: todo o histórico de preços do sistema está, atualmente, ancorado nestas duas lojas genéricas, e não em lojas físicas concretas. Por outro lado, a base de dados de produção contém o catálogo completo de lojas físicas das duas insígnias, com morada e coordenadas reais.

Este catálogo de lojas físicas foi obtido com um scraper dedicado (`all-stores-scrapers`), que recolhe morada e coordenadas de todas as lojas do Continente e do Pingo Doce a partir dos respetivos localizadores de lojas, produzindo um total de 908 lojas físicas (406 do Continente e 502 do Pingo Doce). O resultado da recolha foi aplicado à base de dados através do script Structured Query Language (SQL) de *upsert* gerado pelo próprio scraper, executado manualmente. Esta operação não está, no entanto, codificada como uma migração do Entity Framework Core nem integrada em nenhuma pipeline de CI/CD: é um passo manual e não versionado, pelo que uma nova instância da base de dados criada apenas a partir das migrações do serviço não reproduz o catálogo completo, ficando limitada ao pequeno conjunto

de lojas semeado na migração de sementeira. A lacuna não está, portanto, na recolha de dados de lojas — que já cobre a totalidade das duas insígnias —, mas na reprodutibilidade e automatização desse passo de importação.

Os endpoints expostos são:

- GET /stores — devolve lojas por identificador ou por nome;
- POST /stores — cria uma nova loja;
- PUT /stores/{storeId} — atualiza uma loja;
- DELETE /stores/{storeId} — elimina uma loja.

4.2.5 Account Service

O Account Service gere a conta do utilizador autenticado e funciona como ponto de contacto para funcionalidades transversais, como favoritos e tokens de push. O serviço sincroniza a conta a partir do token JWT emitido pelo Keycloak e mantém informação operacional associada ao utilizador, sem duplicar a lógica de autenticação.

Os endpoints expostos são:

- POST /accounts/me/sync — garante que a conta associada ao token existe na base de dados;
- GET /accounts/me/favorites — devolve os produtos favoritos do utilizador autenticado;
- POST /accounts/me/favorites — adiciona um produto aos favoritos;
- DELETE /accounts/me/favorites/{productId} — remove um produto dos favoritos;
- POST /accounts/me/push-tokens — regista ou atualiza o token FCM do dispositivo.

4.3 Pipeline de Recolha de Dados

A recolha automática de dados de produtos e preços é um dos pilares do ShopISEL, procurando manter a informação apresentada ao utilizador atual e coerente com as fontes recolhidas. Neste fluxo, a expressão *worker* refere-se ao modo de execução pontual de componentes que processam uma tarefa e terminam logo após concluí-la, enquanto os serviços backend permanecem expostos como APIs. Como se pode ver na Figura 4.2, o fluxo é iniciado pelo *Scraper Orchestrator*, segue para os scrapers e termina no *MatchQueue* e na entrega de notificações.

4.3.1 Scraper Orchestrator

O Scraper Orchestrator é um repositório de workflows de GitHub Actions responsável por coordenar a execução dos scrapers de múltiplas insígnias. Cada scraper expõe um workflow reutilizável (*reusable workflow*), que pode ser invocado pelo orchestrator através do mecanismo `workflow_call`.

Esta abordagem permite executar os scrapers em paralelo, aguardar a conclusão de todos eles e, de seguida, desencadear o processamento do MatchQueue. A adição de um novo scraper para uma insígnia adicional requer apenas a referência ao seu workflow reutilizável no orchestrator, sem necessidade de alterar os scrapers existentes.

4.3.2 Scraper Continente

O Scraper Continente é uma aplicação .NET executada como tarefa pontual através de GitHub Actions que utiliza o Playwright para navegar automaticamente no website do Continente e extrair informação de produtos por categoria. O Playwright permite controlar um browser real, sendo adequado para páginas com conteúdo renderizado dinamicamente no cliente, como é o caso do website do Continente [20].

O scraper percorre as categorias configuradas, efetua scroll para carregar mais produtos (mecanismo de lazy loading), e extrai para cada produto o nome, preço atual, preço anterior, preço por unidade, disponibilidade, Uniform Resource Locator (URL) da página e URL da imagem. O resultado é exportado para um conjunto de ficheiros JSON no diretório `output/`: o ficheiro `continente_products.json` contém todos os produtos, sendo ainda gerados ficheiros por grupo de categorias (`continente_fresh_products.json`, `continente_dairy_and_eggs_products.json`, `continente_beverages_products.json`), bem como ficheiros individuais por categoria no sub-diretório `categories/`.

A estrutura do scraper divide as responsabilidades em módulos distintos:

- `Scrapers/`: contém a lógica de navegação e extração por categoria;
- `Parsing/`: realiza o parsing do HTML e a normalização de texto (preços, unidades);
- `Models/`: define as estruturas de dados para fontes de categoria e produtos extraídos;
- `Config/`: configuração das fontes de scraping (categorias e URLs de entrada);
- `Matching/`: lógica de correspondência de produtos entre fontes;
- `Sync/`: sincronização dos dados extraídos com o MongoDB.

4.3.3 Scraper Pingo Doce

O Scraper Pingo Doce segue a mesma abordagem baseada em Playwright, adaptada à estrutura do website do Pingo Doce. Adicionalmente, suporta dois modos de execução:

- **Modo normal**: efetua o scraping ao vivo e persiste os resultados em MongoDB;
- **Modo -from-json**: processa um ficheiro JSON previamente gerado, evitando pedidos desnecessários ao website.

Os dados são persistidos em MongoDB [21] em duas coleções:

- `scrape_runs`: um documento por execução, identificado por um `run_id` único;
- `scrape_products`: um documento por produto por execução, com um identificador estável baseado no URL externo ou numa combinação de categoria e nome.

Esta estratégia de identificação estável permite ao MatchQueue detetar alterações de preço entre execuções consecutivas, comparando os valores pelo mesmo identificador de produto.

O scraper inclui um workflow de GitHub Actions que o executa de forma agendada (cron), lendo as credenciais de acesso ao MongoDB a partir de segredos do repositório.

4.3.4 MatchQueue

O MatchQueue é um serviço ASP.NET Core que expõe uma API própria para desencadear o processamento de correspondência, mas essa via não é utilizada no fluxo operacional do projeto. Na prática, o componente é executado apenas pelo GitHub Actions em modo *worker* após a conclusão dos scrapers; nesse modo, processa o evento `repository_dispatch`, executa a normalização e termina a execução. Este componente faz a ponte entre os dados brutos recolhidos pelos scrapers, guardados em MongoDB, e os dados já preparados para uso no resto do sistema. Para isso, o MatchQueue usa diretamente as mesmas tabelas do Products Service, do Prices Service e do Account Service, em vez de guardar cópias separadas. Na prática, isto significa que o MatchQueue lê e escreve nas tabelas `products`, `categories`, `prices` e `favorite_products` para registar o resultado do processamento.

Esta solução funciona, mas exige cuidado: se um serviço alterar a estrutura de uma tabela, os outros têm de acompanhar essa mudança manualmente. Um exemplo é a coluna `canonical_key` da tabela `products`, que foi adicionada pelo MatchQueue e não é usada pelo Products Service. Outro exemplo é a tabela `categories`: o MatchQueue apenas usa o campo `Id`, enquanto o Products Service é responsável por `name`, `image` e `parent_category_id`. Por isso, cada serviço trabalha apenas com uma parte da mesma tabela.

Além destas entidades partilhadas, o MatchQueue tem uma entidade própria, sem equivalente noutro serviço:

- **NotificationEnqueueEntity**: fila de notificações pendentes, com referência ao utilizador, ao produto e ao estado de processamento (`Checked`), persistida na tabela `notification_enque`, usada para determinar quais os utilizadores a notificar quando um produto favorito desce de preço.

O MatchQueue é orquestrado por um workflow de GitHub Actions que corre após a conclusão dos scrapers, permitindo encadear a recolha, a normalização e o processamento de notificações como uma etapa temporária da pipeline. Nesta fase, o GitHub Actions funciona como camada de orquestração e não como o tipo de aplicação em si.

4.4 FCM Notification Worker

O FCM Notification Worker é uma aplicação .NET de execução pontual (*run-to-completion*): ao contrário dos serviços de backend, não possui imagem Docker nem deployment persistente, sendo compilada e executada diretamente num *runner* do GitHub Actions sempre que recebe o

evento `repository_dispatch` disponibilizado automaticamente pelo workflow do MatchQueue ao concluir o seu processamento (podendo também ser executada manualmente). O Firebase Cloud Messaging (Firebase Cloud Messaging (FCM)) [8] é um serviço de entrega de mensagens push mantido pela Google, que permite enviar notificações para dispositivos Android e iOS de forma fiável e escalável. O seu funcionamento segue os seguintes passos:

1. Inicializa o cliente Firebase com as credenciais, guardadas como segredo e codificadas em Base64 apenas para permitir a sua injeção como variável de ambiente de texto simples (o Base64 não constitui, por si só, um mecanismo de proteção);
2. Estabelece ligação à base de dados PostgreSQL do MatchQueue;
3. Consulta a tabela `notification_enqueue` para obter notificações pendentes não processadas (`checked = false`), cruzando com a tabela `account_push_tokens` para obter os tokens FCM ativos dos utilizadores;
4. Constrói e envia as mensagens push em lotes de 500 (limite imposto pelo Firebase);
5. Marca as notificações processadas como `checked = true`.

As notificações enviadas informam o utilizador de que o preço de um produto favorito desceu, incluindo nos dados da mensagem o identificador do produto e da notificação, permitindo à aplicação móvel navegar diretamente para o produto relevante.

4.5 Aplicação Web

A aplicação web do ShopISEL é desenvolvida em React com TypeScript e Vite, proporcionando uma interface moderna e responsiva para preparação e análise de compras num ambiente de desktop.

A estrutura da aplicação segue uma organização por responsabilidade:

- `src/api/`: camada de comunicação com os serviços de backend, abstraindo os detalhes de cada pedido HTTP;
- `src/auth/`: integração com o Keycloak para autenticação, gestão do ciclo de vida do token e redirecionamentos;
- `src/app/`: componentes de página e lógica de navegação;
- `src/styles/`: estilos globais e configuração de temas.

A interface recorre a bibliotecas de componentes como Material UI, Radix UI e Tailwind CSS, que permitem construir ecrãs consistentes e acessíveis com menor esforço de implementação. O TypeScript garante a verificação estática dos tipos trocados com as APIs, reduzindo erros em tempo de execução.

A aplicação web é servida em produção através de um servidor Nginx, sendo a imagem Docker publicada automaticamente pelo pipeline de CI/CD.

4.5.1 Integração com Concorrência Otimista

A aplicação web inclui suporte para concorrência otimista através da integração com o token de versão exposto pelo contrato do *List Service*. A camada de comunicação com a API modela o campo `version` em cada lista, incluindo-o de forma automática no cabeçalho (*header*) `If-Match` em todos os pedidos de atualização e eliminação.

Os ecrãs de gestão de listas — nomeadamente a página de detalhe, o ecrã inicial, a listagem geral e o painel de adição de produtos — asseguram a propagação da versão corrente em cada operação de escrita. Sempre que ocorrem ações como a alteração do nome, marcação de itens, ou a adição e remoção de produtos, a aplicação submete a versão correspondente ao momento do carregamento inicial dos dados.

Caso o *backend* detete que a lista foi modificada concorrentialmente por outro cliente, o pedido é rejeitado com o código de estado 409. Nestes cenários, a interface apresenta uma mensagem informativa que indica que a lista foi alterada noutro dispositivo, instruindo o utilizador a recarregar os dados antes de redefinir a operação. Este mecanismo garante a integridade dos dados e previne a perda silenciosa de informação em cenários de acesso concorrente.

4.5.2 Arquitetura white-label dinâmica

Para permitir a publicação da aplicação web sob diferentes identidades visuais sem manter ramificações de código separadas, foi implementado um sistema de personalização em tempo de *build*. O objetivo é alterar nome, slogan, logótipo e esquema de cores da marca apenas através de parâmetros injetados no processo de construção e *deployment*, mantendo o código-fonte partilhado focado na lógica funcional da aplicação.

A configuração de marca concentra-se num módulo central que lê variáveis de ambiente prefixadas com `VITE_` e aplica valores por defeito correspondentes à identidade Shopisel quando nenhuma personalização é fornecida. Para além dos elementos textuais e do logótipo, o módulo calcula automaticamente tons derivados — versões mais claras, mais escuras e com transparência — a partir das cores primária e secundária indicadas, evitando que cada variante exija a definição manual de dezenas de valores cromáticos.

No arranque da aplicação, as cores resolvidas são injetadas como propriedades CSS personalizadas no documento, o que permite que as classes utilitárias do Tailwind e os componentes existentes adotem imediatamente a nova paleta. Elementos como a barra lateral, o ecrã de *splash*, o *onboarding* e os controlos de navegação passam a refletir a marca configurada sem alterações estruturais ao código.

O fluxo completo de personalização percorre o pipeline de CI/CD: o workflow de publicação Docker suporta execução manual com inputs opcionais para os parâmetros de marca; estes valores são passados como argumentos de construção da imagem, convertidos em variáveis de ambiente durante o *build* do Vite e incorporados de forma estática no *bundle* final. Quando o workflow corre automaticamente após integração no branch principal, os inputs as-

sumem valores vazios e a aplicação mantém a identidade por defeito. Desta forma, a ativação de uma nova variante white label reduz-se a executar o workflow de *deploy* com os parâmetros desejados, sem necessidade de editar o repositório.

4.6 Aplicação Móvel

A aplicação móvel é desenvolvida em React Native com o framework Expo, permitindo uma única base de código para Android e iOS. O Expo Router organiza a navegação por ficheiros e separa o fluxo principal em cinco separadores: *Home*, *Lists*, *Scan*, *Prices* e *Profile*. Para além destes separadores, existem ainda ecrãs dedicados a *Onboarding*, *Auth* e *Alerts*, que são apresentados no *stack* principal da aplicação.

4.6.1 Leitura de código de barras

O ecrã de leitura de código de barras é um dos pontos mais práticos da experiência móvel. A aplicação usa a câmara do dispositivo para capturar o código, extrai o identificador e complementa a informação do produto com o serviço público Open Food Facts quando o catálogo local não é suficiente. De seguida, envia ao endpoint `/products/qrCode` um corpo JSON com o `barcode` lido e as `keywords` obtidas a partir dos dados complementares, permitindo ao backend tentar primeiro uma correspondência exata e, se necessário, devolver produtos relacionados [9, 23].

O fluxo implementado distingue três resultados distintos no ecrã de scan: produto encontrado, apenas produtos relacionados e nenhum resultado. Quando existe um produto exato, a aplicação apresenta a ação de adicionar à lista, abre uma modal para escolher a lista de destino e guarda o produto lido localmente para que possa ser reutilizado no separador *Prices* quando não exista `productId` na rota. Quando surgem apenas produtos relacionados, estes são apresentados como itens clicáveis, sem botões adicionais desnecessários. Se não for encontrado qualquer resultado, o utilizador recebe uma mensagem de falha e pode tentar novamente ou pesquisar manualmente pelo nome do produto.

Este fluxo pede autorização explícita para aceder à câmara quando o ecrã de scan é aberto e a permissão ainda não foi concedida. A interface apresenta um estado próprio de permissões e só ativa a leitura depois de o utilizador autorizar o acesso, garantindo que o scan funciona de forma controlada e transparente.

4.6.2 Localização e preços próximos

A componente de preços próximos utiliza o módulo de localização do Expo para obter a posição do utilizador. Como o preço devolvido pelo Prices Service está associado a uma das duas lojas genéricas por insígnia (Secção 4.2.4), a aplicação usa a insígnia dessa loja para procurar, entre as lojas físicas com coordenadas da mesma insígnia, as três mais próximas do utilizador, e apresenta o preço sob o nome e morada dessas lojas. Ou seja, a proximidade

determina que loja física é mostrada ao utilizador, mas não afeta o valor do preço apresentado, que é o mesmo em todas as lojas da insígnia. Quando não existem coordenadas disponíveis — do utilizador ou de lojas físicas dessa insígnia — a aplicação apresenta a loja genérica, sem indicação de distância. O objetivo não foi criar um sistema de georreferenciação complexo no servidor, mas sim aproveitar a localização do dispositivo para dar contexto à consulta de preços e destacar lojas próximas no momento da compra.

A obtenção da localização também depende de autorização do utilizador: o *hook* de localização pede permissão em *foreground* antes de obter e acompanhar a posição atual. Quando a permissão é recusada, a aplicação mostra essa limitação de forma explícita e continua a funcionar, apenas sem ordenar os preços por proximidade.

A organização do código fonte por domínio funcional facilita a manutenção e a extensão da aplicação:

- `src/api/`: chamadas aos serviços de backend;
- `src/auth/`: gestão do token de autenticação com armazenamento seguro;
- `src/push/`: registo do token FCM e integração com notificações push;
- `src/location/`: obtenção da localização do utilizador;
- `src/favorites/`: gestão dos produtos favoritos do utilizador;
- `src/i18n/`: suporte a múltiplos idiomas;
- `src/theme/`: sistema de temas e estilos globais.

O NativeWind é utilizado para estilização, aplicando as utilidades do Tailwind CSS ao contexto React Native [16], garantindo consistência visual com a aplicação web. O armazenamento seguro dos tokens de autenticação utiliza o Expo Secure Store, que persiste dados de forma cifrada no dispositivo.

4.7 Autenticação e Autorização

A autenticação e autorização no ShopISEL é centralizada no Keycloak, uma plataforma de gestão de identidade e acesso que implementa os protocolos OIDC e OAuth 2.0.

O fluxo de autenticação segue o padrão Authorization Code Flow com PKCE (Proof Key for Code Exchange), adequado tanto para aplicações web como para aplicações móveis. O utilizador é redirecionado para o servidor Keycloak para introduzir as suas credenciais; após autenticação bem-sucedida, é emitido um token de acesso JWT que é enviado em cada pedido aos serviços protegidos. A Figura 4.4 ilustra este fluxo.

Os serviços que requerem autenticação (List Service e Account Service) validam o token da seguinte forma:

1. Verificam que o emissor (`iss`) corresponde ao realm Keycloak configurado;
2. Verificam que o token não expirou (`exp`);
3. Verificam que a chave de assinatura corresponde à chave pública do Keycloak (obtida via OIDC Discovery);

4. Verificam que o claim `azp` (authorized party) corresponde a um dos clientes autorizados (web, mobile).

Esta abordagem evita que cada serviço implemente autenticação própria, centraliza a gestão de utilizadores no Keycloak e permite que novos serviços se integrem facilmente no ecossistema de autenticação existente.

No registo de uma nova conta, o Keycloak exige a confirmação do endereço de email antes de permitir o primeiro login, através da *required action* nativa *Verify Email*: após a criação da conta, é enviado automaticamente um email de confirmação com uma ligação que valida o endereço e desbloqueia o acesso. O envio deste email é feito através do Brevo, configurado como servidor Simple Mail Transfer Protocol (SMTP) do realm do Keycloak, sem necessidade de código adicional nos serviços do ShopISEL.

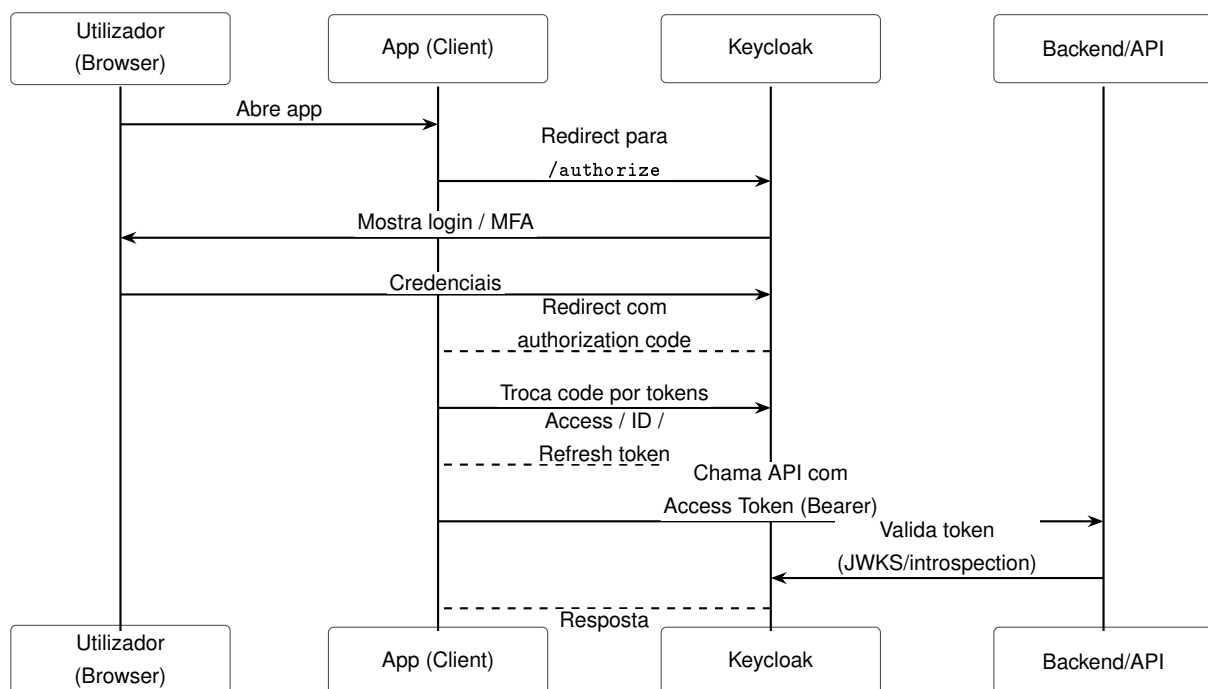


Figura 4.4 Fluxo de autenticação com Keycloak

4.8 Gateway e Infraestrutura de Deployment

O acesso às APIs dos serviços de backend é feito através de um gateway Nginx, que atua como ponto de entrada único para as aplicações cliente. O Nginx encaminha os pedidos para o serviço correto com base no caminho do URL:

- `/api/lists/...` → List Service;
- `/api/products/...` → Products Service;
- `/api/categories/...` → Products Service (categorias);
- `/api/prices/...` → Prices Service;
- `/api/stores/...` → Stores Service;

- `/api/accounts/...` → Account Service;
- `/` → Aplicação Web (frontend).

Esta arquitetura de gateway simplifica a configuração das aplicações cliente, que comunicam com um único endpoint base, e permite adicionar funcionalidades transversais como rate limiting, logging centralizado ou cache HTTP sem modificar os serviços individuais.

Todo o sistema é containerizado com Docker. Cada serviço dispõe de um Dockerfile que produz uma imagem mínima e otimizada para produção. Os serviços são implantados no Fly.io, tirando partido dos mecanismos de deployment e operação disponibilizados pela plataforma.

A configuração de deployment está declarada nos ficheiros de infraestrutura e nos pipelines de CI/CD, que publicam as imagens no GitHub Container Registry e automatizam o processo de entrega.

Capítulo 5

Processo de Desenvolvimento e Integração Contínua

Este capítulo descreve o processo de desenvolvimento adotado no projeto ShopISEL, incluindo a organização dos repositórios, as práticas de controlo de versões, as pipelines de integração e entrega contínua (CI/CD) e a estratégia de deployment automatizado.

5.1 Organização do Projeto em Repositórios

O ShopISEL adota uma estrutura multi-repositório, em que cada serviço ou componente principal reside num repositório Git independente. Esta abordagem permite que cada equipa ou componente evolua autonomamente, com o seu próprio ciclo de testes, construção e publicação, sem introduzir dependências de build entre serviços distintos.

Os repositórios principais do projeto são:

- `list-service` — serviço de listas de compras;
- `products-service` — serviço de produtos e categorias;
- `prices-service` — serviço de preços;
- `stores-service` — serviço de lojas e georreferenciação;
- `account-service` — serviço de perfil de utilizador;
- `matchqueue` — componente worker de correspondência e normalização de dados;
- `fcm-notification-worker` — worker de notificações push;
- `scraper-continente` — scraper do website Continente;
- `scraper-pingodoce` — scraper do website Pingo Doce;
- `scraper-orchestrator` — orchestrator dos scrapers via GitHub Actions;
- `all-stores-scraper` — scraper das moradas e coordenadas das lojas físicas do Continente e do Pingo Doce;
- `frontend-web` — aplicação web React;
- `frontend-mobile` — aplicação móvel React Native / Expo;

- `deploy` — configuração de infraestrutura e gateway Nginx;
- `keycloak` — configuração do servidor Keycloak.

5.2 Controlo de Versões e Estratégia de Branches

O controlo de versões é assegurado pelo Git, com os repositórios alojados no GitHub. O branch principal de cada repositório é o `main`, que representa sempre um estado estável e implantável do serviço. O desenvolvimento de novas funcionalidades é realizado em branches separados, que são integrados no `main` através de Pull Requests após revisão de código.

Esta prática permite que a pipeline de CI/CD, configurada no branch `main`, seja ativada apenas quando o código está pronto para integração, garantindo que apenas código revisto e testado é publicado e implantado.

5.3 Pipelines de Integração e Entrega Contínua

Cada repositório de backend inclui uma pipeline de CI/CD implementada com GitHub Actions. As pipelines são ativadas automaticamente em cada push para o branch `main` e seguem, tipicamente, as seguintes etapas:

1. **Build e teste:** o código é compilado e os testes automatizados são executados. Esta etapa garante que o código integrado compila sem erros e que os testes de integração passam antes de avançar para as etapas seguintes.
2. **Análise de qualidade:** em alguns serviços, é realizada uma análise estática de código através do SonarCloud, que reporta métricas de qualidade, vulnerabilidades e código duplicado.
3. **Construção da imagem Docker:** a imagem Docker do serviço é construída com base no Dockerfile do repositório.
4. **Publicação da imagem:** a imagem construída é publicada no GitHub Container Registry (`ghcr.io/shopisel/<service-name>:latest`), ficando disponível para o processo de deployment.
5. **Deploy automático:** a plataforma de hosting (Fly.io) é configurada para detetar novas versões da imagem e realizar o deployment de forma automática.

Para os scrapers, as pipelines incluem uma etapa adicional de execução agendada por cron, que dispara automaticamente o processo de recolha de dados em intervalos regulares, sem necessidade de intervenção manual.

A Figura 5.1 ilustra o fluxo geral de CI/CD para os serviços de backend.

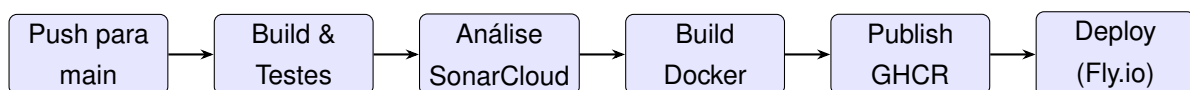


Figura 5.1 Fluxo de CI/CD para os serviços de backend

5.4 Reusable Workflows e Orquestração de Scrapers

Um aspecto relevante do processo de desenvolvimento é a utilização de GitHub Actions reusable workflows para a orquestração dos scrapers. Cada repositório de scraper expõe o seu processo de execução como um workflow reutilizável (`workflow_call`), que pode ser invocado por repositórios externos.

O repositório `scraper-orchestrator` contém um workflow, ativado diariamente por *cron* ou manualmente, que:

1. Invoca em paralelo os workflows reutilizáveis de cada scraper;
2. Aguarda a conclusão de todos os scrapers (`needs: [...]`);
3. Despoleta, através de um evento `repository_dispatch`, o workflow do MatchQueue, que corre a aplicação em modo *worker* para normalizar os dados recolhidos.

Ao concluir com sucesso, o workflow do MatchQueue disponibiliza, por sua vez, um novo evento `repository_dispatch` para o repositório `fcm-notification-worker`, que compila e executa a aplicação diretamente no *runner*, sem deployment persistente, encaminhando as notificações pendentes para o Firebase Cloud Messaging. Esta cadeia de eventos entre repositórios substitui a necessidade de um agendador central: o GitHub Actions apenas orquestra a execução, enquanto cada componente mantém o seu papel funcional.

Esta arquitetura de orquestração permite adicionar suporte a uma nova insígnia simplesmente criando um novo repositório de scraper com um workflow reutilizável e referenciando-o no orchestrator, sem modificar nenhum dos scrapers existentes.

5.5 Ferramentas de Suporte ao Desenvolvimento

O desenvolvimento do ShopISEL recorreu a um conjunto de ferramentas complementares que contribuíram para a qualidade e eficiência do processo:

- **GitHub Copilot:** ferramenta de assistência à escrita de código baseada em modelos de linguagem de grande dimensão (*Large Language Models*), utilizada para sugestões de código, geração de testes e validação de lógica de negócio.
- **SonarCloud:** plataforma de análise estática integrada nas pipelines de CI/CD, que reporta métricas de qualidade de código, vulnerabilidades de segurança e cobertura de testes.
- **Docker:** utilizado para containerizar todos os serviços, garantindo reprodutibilidade dos ambientes de desenvolvimento, teste e produção.
- **GitHub Container Registry:** repositório de imagens Docker integrado no GitHub, utilizado para publicar e versionar as imagens de cada serviço.
- **Fly.io:** plataforma de cloud hosting utilizada para o deployment dos serviços, com suporte nativo a imagens Docker e deployment automático por webhook.

5.6 Gestão de Configuração e Segredos

A configuração de cada serviço é injetada por variáveis de ambiente, seguindo o princípio dos 12-factor apps. Isto permite que a mesma imagem Docker seja executada em diferentes ambientes (desenvolvimento, staging, produção) com configurações distintas, sem necessidade de recompilação.

Os segredos sensíveis (cadeias de ligação a bases de dados, credenciais do Keycloak, credenciais Firebase, tokens de acesso ao MongoDB) são geridos como GitHub Secrets nos repositórios correspondentes, sendo injetados nas pipelines de CI/CD e nos ambientes de produção sem exposição no código fonte.

Esta prática é particularmente relevante para os scrapers, cujos workflows de GitHub Actions necessitam de acesso às credenciais do MongoDB e às configurações do MatchQueue para funcionar corretamente.

5.7 Desenvolvimento White Label

O conceito de white label aplicado ao ShopISEL consiste em disponibilizar a aplicação web sob diferentes identidades visuais — nome, slogan, logótipo e esquema de cores — adaptadas a contextos de retalho ou parceiros comerciais distintos, sem manter bases de código separadas nem alterar manualmente ficheiros de configuração no repositório.

A solução adotada no repositório `frontend-web` baseia-se na injeção de parâmetros de marca em tempo de *build*, acoplando três camadas do processo de entrega: GitHub Actions, Docker e Vite. O workflow de publicação da imagem Docker foi estendido para suportar execução manual (`workflow_dispatch`), permitindo introduzir seis parâmetros opcionais — nome da marca, slogan, slogan da barra lateral, URL do logótipo e cores primária e secundária. Quando o workflow é disparado automaticamente por integração contínua no branch principal, estes parâmetros ficam vazios e a aplicação assume a identidade Shopisel por defeito.

Durante a construção da imagem, os valores recebidos são passados como argumentos de *build* do Docker e convertidos em variáveis de ambiente com o prefixo `VITE_`. O Vite, ferramenta de *build* do frontend, incorpora estas variáveis de forma estática no *bundle* JavaScript final, substituindo as referências no código-fonte no momento da compilação. Um módulo de configuração central resolve os valores finais da marca, aplicando *fallbacks* quando algum parâmetro não é fornecido, e calcula tons complementares a partir das cores base.

Em ambiente de desenvolvimento local, as mesmas variáveis podem ser definidas num ficheiro de ambiente, embora seja necessário reiniciar o servidor de desenvolvimento após alterações, uma vez que a injeção do Vite ocorre no arranque. Para valores hexadecimais de cor, estes devem ser colocados entre aspas no ficheiro local, dado que o carácter `#` seria interpretado como início de comentário pelo *parser* de variáveis de ambiente.

O resultado desta abordagem é um modelo de *deployment* sem alteração de código: um gestor ou operador pode ativar uma nova variante de marca executando manualmente o workflow de publicação com os parâmetros desejados, produzindo uma imagem Docker pronta

para implantação. Toda a infraestrutura de testes, construção e entrega existente é reutilizada, e o código-fonte permanece centrado na funcionalidade da aplicação e na marca por defeito.

Capítulo 6

Testes e Avaliação

A fase de testes e avaliação é parte integrante do processo de desenvolvimento, permitindo validar o comportamento do sistema, identificar regressões e garantir a conformidade dos contratos das APIs. Este capítulo descreve a estratégia de testes adotada no ShopISEL, os mecanismos de isolamento utilizados e os resultados obtidos.

6.1 Estratégia de Testes

O ShopISEL adota uma abordagem de testes de integração de ponta a ponta (end-to-end) ao nível de cada serviço, em detrimento de testes unitários isolados. Esta opção justifica-se pela natureza dos serviços: dado que a maioria da lógica de negócio se manifesta na interação entre endpoints HTTP, persistência em base de dados e validação de autenticação, os testes de integração oferecem um nível de confiança superior sobre o comportamento real do sistema.

Os testes são implementados com o framework xUnit [27], amplamente adotado no ecossistema .NET, e recorrem à classe `WebApplicationFactory<Program>` da ASP.NET Core para inicializar o serviço em memória, sem necessidade de servidores externos [19].

Os serviços com cobertura de testes de integração são:

- **List Service** (`ListService.Tests`);
- **Products Service** (`ProductService.Tests`);
- **Prices Service** (`PricesService.Tests`);
- **Account Service** (`AccountService.Tests`), cobrindo sincronização de conta, favoritos e registo de tokens de push.

O **Stores Service** tem um projeto de testes criado, mas o seu conjunto de casos é mais reduzido do que o dos restantes serviços centrais. Na prática, isso significa que a cobertura automatizada está mais madura nos domínios onde existe mais lógica de negócio, enquanto os serviços mais simples são validados sobretudo de forma funcional. Ambos os grupos são também exercitados indiretamente durante a validação funcional descrita na Secção 6.5.

A análise estática de código realizada pelo SonarCloud nas pipelines de CI/CD comple-

menta os testes automatizados, reportando métricas de qualidade, vulnerabilidades e cobertura de código para os serviços que possuem projetos de teste.

6.2 Infraestrutura de Testes

Para que os testes de integração possam ser executados de forma isolada e reprodutível, sem depender de serviços externos como PostgreSQL em produção ou Keycloak, são utilizados dois mecanismos de substituição:

6.2.1 Base de dados em memória com SQLite

A classe `ListServiceApiFactory` (e equivalentes nos outros serviços) substitui o contexto de base de dados da aplicação por uma instância SQLite com ficheiro temporário. Esta instância é partilhada por todos os casos de teste de uma mesma classe através de `IClassFixture`, sendo recriada do zero (`EnsureDeleted` seguido de `EnsureCreated`) no arranque de cada classe de testes. Esta abordagem garante que:

- Cada classe de testes parte de um estado limpo e previsível, uma vez que a base de dados é recriada antes da primeira execução;
- Não é necessário provisionar uma instância de PostgreSQL para executar os testes localmente ou nas pipelines de CI/CD;
- Os testes são deterministas na sua avaliação individual, embora, por partilharem a mesma instância de base de dados dentro da classe, não sejam totalmente independentes entre si ao nível da persistência: dados criados por um caso de teste permanecem visíveis para os casos seguintes da mesma classe. Esta limitação é mitigada pelo facto de as asserções verificarem a presença ou ausência de entidades específicas (`Assert.Contains/Assert.DoesNotContain`), em vez de contagens exatas, tornando os testes tolerantes a este estado partilhado.

Referimos ainda que o ficheiro SQLite temporário criado para cada classe de testes não é explicitamente removido do sistema de ficheiros no final da execução, dado não existir libertação de recursos que invoque `File.Delete` sobre o ficheiro subjacente. Trata-se de uma limitação conhecida desta abordagem, sem impacto na correção dos testes, mas que pode originar acumulação de ficheiros residuais no diretório temporário do sistema operativo ao longo de sucessivas execuções, e que deveria ser endereçada numa iteração futura.

Importa ainda notar que o SQLite não reproduz de forma exata o comportamento do PostgreSQL utilizado em produção: existem diferenças de dialeto SQL, de funções e tipos suportados, e de semântica de concorrência entre os dois motores. Os testes de integração validam, por isso, o comportamento funcional dos endpoints e a lógica de negócio da aplicação, mas não garantem por si só que o mesmo comportamento se replica de forma idêntica sobre PostgreSQL em todos os cenários.

6.2.2 Handler de autenticação de teste

Dado que os endpoints protegidos requerem um token JWT válido emitido pelo Keycloak, os testes substituem o esquema de autenticação real por um handler personalizado (`TestAuthHandler`). Este handler autentica automaticamente cada pedido HTTP de teste com um utilizador fixo, permitindo testar o comportamento dos endpoints sem necessidade de uma instância Keycloak em execução.

Para testar cenários de isolamento entre utilizadores, é possível criar clientes HTTP adicionais com identidades distintas, modificando o header de utilizador de teste (`X-Test-User`). Esta técnica é utilizada nos testes de isolamento descritos na Secção 6.3.

6.3 Casos de Teste do List Service

O conjunto de testes do List Service cobre os principais fluxos de negócio e requisitos de segurança do serviço.

6.3.1 Fluxo CRUD completo

O teste `ListsCrudFlow_WorksEndToEnd` valida o ciclo de vida completo de uma lista de compras:

1. **Criação:** submete um pedido `POST /lists` com nome e dois itens. Verifica que a resposta tem código HTTP 201, que o identificador da lista começa com o prefixo `list_`, que os dois itens foram criados com identificadores positivos e que a lista inclui um token de versão válido.
2. **Consulta:** faz `GET /lists/{id}` e verifica que o nome da lista corresponde ao submetido.
3. **Atualização:** submete `PUT /lists/{id}` com novo nome, um único item diferente e o header `If-Match` com a versão obtida na criação. Verifica que a resposta reflete as alterações corretamente e que a versão da lista foi atualizada.
4. **Eliminação:** faz `DELETE /lists/{id}` com o header `If-Match` correspondente à versão mais recente e verifica que a resposta é HTTP 204. Subsequentemente, um `GET /lists/{id}` deve retornar HTTP 404.

6.3.2 Isolamento por proprietário

O teste `ListOwnerIsolation_OtherUserCannotAccessList` verifica que um utilizador não consegue aceder às listas de outro utilizador:

1. O utilizador A cria uma lista;
2. Um cliente HTTP configurado com a identidade do utilizador B tenta obter e eliminar a lista;

3. Verifica-se que ambas as operações retornam HTTP 404, confirmando que o isolamento por `OwnerId` funciona corretamente.

Este teste é fundamental para garantir que a privacidade dos dados do utilizador é respeitada, mesmo perante pedidos autenticados de outros utilizadores.

6.3.3 Filtragem no GET /lists

O teste `GetAll_ReturnsOnlyListsFromAuthenticatedUser` verifica que o endpoint `GET /lists` devolve apenas as listas do utilizador autenticado:

1. O utilizador A e o utilizador B criam, cada um, uma lista;
2. O utilizador A executa `GET /lists` e verifica que a resposta contém a sua lista mas não a do utilizador B.

6.3.4 Conflito de concorrência

O teste `Update_WithStaleVersion_ReturnsConflict` valida o mecanismo de concorrência otimista quando duas atualizações concorrem sobre a mesma lista:

1. Cria uma lista e regista a versão devolvida na resposta;
2. Executa uma primeira atualização bem-sucedida com o header `If-Match` correto, obtendo uma nova versão;
3. Submete uma segunda atualização reutilizando deliberadamente a versão original, já obsoleta;
4. Verifica que o serviço responde com HTTP 409 (*Conflict*), confirmando que alterações concorrentes não sobrepõem uns aos outros de forma silenciosa.

Este teste garante que o contrato de versão funciona de ponta a ponta — desde a persistência com token de concorrência até à tradução do conflito numa resposta HTTP previsível para as aplicações cliente.

6.4 Execução dos Testes nas Pipelines de CI/CD

Os testes são integrados nas pipelines de CI/CD de cada serviço. Na etapa de *build e teste*, o comando `dotnet test` é executado, correndo todos os testes de integração do projeto. Se algum teste falhar, a pipeline é interrompida e o código não avança para a etapa de construção da imagem Docker, impedindo que código com regressões seja publicado ou implantado.

Esta integração garante que qualquer alteração ao código que quebre um contrato de API ou introduza uma regressão de comportamento é detetada automaticamente, antes de chegar ao ambiente de produção.

6.5 Avaliação Funcional

Para além dos testes automatizados, o sistema foi sujeito a validação funcional manual, que permitiu verificar o comportamento integrado de todos os componentes em conjunto — aspeto que os testes de integração por serviço não cobrem, dado que cada serviço é testado de forma isolada.

A validação funcional incluiu os seguintes cenários:

- **Registo e autenticação:** criação de conta no Keycloak, login a partir da aplicação web e da aplicação móvel, renovação de token e logout;
- **Gestão de listas:** criação, edição e eliminação de listas de compras na aplicação web e na aplicação móvel, com verificação da sincronização entre dispositivos e do tratamento de conflitos quando a mesma lista é editada em simultâneo a partir de clientes diferentes;
- **Personalização white label:** execução manual do workflow de publicação com parâmetros de marca distintos e verificação de que a aplicação implantada reflete corretamente o nome, logótipo e cores configurados;
- **Pesquisa de produtos:** pesquisa por nome e por categoria, consulta de preços por loja e identificação por código de barras;
- **Leitura de código de barras:** captura pela câmara na aplicação móvel, consulta ao backend e sugestão de produto ou alternativas relacionadas;
- **Georreferenciação:** pedido de localização no dispositivo e ordenação dos resultados por proximidade;
- **Notificações push:** registo do token FCM no arranque da aplicação móvel e receção de alertas de descida de preço;
- **Pipeline de scraping:** execução dos scrapers, verificação dos dados no MongoDB, processamento pelo MatchQueue em modo *worker* via GitHub Actions e consulta dos preços atualizados na aplicação.

A validação funcional indicou conformidade do sistema com os principais fluxos definidos nos requisitos, sem se ter realizado uma medição sistemática de desempenho ou testes de carga; a avaliação de tempos de resposta limitou-se à observação qualitativa durante os cenários manuais descritos acima.

Capítulo 7

Trabalho Futuro

Ao longo do desenvolvimento do ShopISEL foram identificados diversos temas e melhorias que, por restrições de tempo, não foi possível explorar ou implementar no âmbito deste projeto. Este capítulo agrupa essas direções de evolução futura por área de incidência: evolução funcional da plataforma, recolha de dados e inteligência artificial, resiliência e infraestrutura, e distribuição da aplicação móvel.

7.1 Evolução Funcional

O ShopISEL foi desenvolvido com uma arquitetura extensível, que facilita a adição de novas funcionalidades e o alargamento do âmbito do sistema. As principais direções de evolução funcional identificadas são:

Histórico de preços e análise de tendências. Atualmente, o sistema mantém o preço mais recente de cada produto por loja. A extensão do modelo de dados para preservar o histórico completo de variações permitiria apresentar ao utilizador gráficos de evolução de preço e detetar padrões sazonais.

Agregação de notificações por utilizador. Atualmente, cada descida de preço de um produto favorito gera uma notificação push independente: um utilizador com, por exemplo, 100 produtos favoritos que desçam de preço no mesmo ciclo de processamento do MatchQueue recebe 100 notificações separadas, em vez de uma única notificação agregada. Uma extensão natural seria introduzir um mecanismo de *buffer* de notificações por utilizador, agrupando as descidas de preço pendentes de uma mesma conta e consolidando-as numa única notificação-resumo (por exemplo, “5 produtos favoritos desceram de preço”), reduzindo o ruído para o utilizador sem perder a informação relevante.

Recomendações personalizadas. Com base nos produtos favoritos, no histórico de listas e nas preferências do utilizador, o sistema poderia oferecer sugestões de produtos ou de lojas mais vantajosas, incorporando lógica de recomendação personalizada.

Partilha de listas. A funcionalidade de partilha de listas de compras entre utilizadores (e.g., membros de uma família) é uma extensão natural do modelo atual, que requer a adição de um mecanismo de partilha e controlo de acesso ao nível do List Service.

Digitalização de talões. A leitura e análise de talões de compra por Optical Character Recognition (OCR) permitiria ao utilizador registar automaticamente o que comprou, o preço pago e a loja, construindo um histórico de compras pessoal e alimentando o sistema com dados de preços reais praticados.

Extensão do white label. A arquitetura white-label dinâmica já implementada na aplicação web pode ser alargada à aplicação móvel, replicando o mesmo princípio de injeção de parâmetros de marca no processo de *build*. Outras evoluções naturais incluem suportar mais elementos configuráveis — como tipografia ou textos de *onboarding* — e automatizar a criação de variantes a partir de um painel de gestão, em vez de depender exclusivamente da execução manual do workflow de publicação.

7.2 Recolha de Dados e Inteligência Artificial

A qualidade dos dados recolhidos é central ao valor do ShopISEL, e foram identificadas várias direções para a tornar mais robusta e abrangente.

Suporte a novas insígnias. A adição de novos scrapers para outras cadeias de supermercados portuguesas (e.g., Auchan, Lidl, Intermarché) é a extensão mais natural do sistema, não requerendo alterações à infraestrutura central. Cada nova insígnia corresponde a um novo repositório de scraper com um workflow reutilizável registado no orchestrator.

Automatização da importação do catálogo de lojas físicas. O scraper dedicado `all-stores-scraper` já recolhe morada e coordenadas da totalidade das lojas físicas do Continente e do Pingo Doce (908 lojas), e o resultado dessa recolha foi aplicado à base de dados de produção através do script SQL de *upsert* que o próprio scraper gera. Esta importação foi, no entanto, executada manualmente, fora de qualquer migração do Entity Framework Core ou pipeline de CI/CD, pelo que não é reproduzível a partir do histórico de migrações nem se repete automaticamente quando as lojas físicas mudam de morada ou abrem novas lojas. Integrar o `all-stores-scraper` numa pipeline periódica, com importação automática para o Stores Service, eliminaria esta dependência de um passo manual e manteria o catálogo de lojas atualizado ao longo do tempo.

Melhorias futuras no scraper. Os scrapers desenvolvidos cumprem o seu propósito, mas dependem da estrutura atual das páginas do Continente e do Pingo Doce, que pode mudar sem aviso. Trabalho futuro nesta área inclui mecanismos de deteção automática de falhas de extração (e.g., alertas quando o número de produtos recolhidos cai abruptamente face ao habitual), maior cobertura de categorias de produtos e a otimização do tempo de execução dos scrapers.

Utilização de Inteligência Artificial (IA) na deteção de produtos. A normalização e correspondência de produtos entre insígnias é atualmente feita com regras de matching desenvolvidas manualmente no MatchQueue. A integração de técnicas de inteligência artificial — como modelos de *embeddings* de texto ou de visão computacional sobre as imagens dos produtos — poderia melhorar significativamente a precisão da deteção de produtos equivalen-

tes entre diferentes lojas, reduzindo a necessidade de regras manuais e facilitando a adição de novas insígnias.

7.3 Resiliência e Infraestrutura

A arquitetura atual cobre os fluxos principais do sistema, mas foram identificadas lacunas de robustez que não chegaram a ser endereçadas.

Tratamento de falhas de serviços indisponíveis. Atualmente, se um dos serviços de backend ficar indisponível, os pedidos que dele dependem falham sem um tratamento uniforme do erro, podendo resultar em respostas pouco claras para as aplicações cliente. Uma direção de evolução natural passa pela introdução de padrões de resiliência — como *circuit breaker*, *retry* com *backoff* e respostas de erro estruturadas e consistentes — para que a indisponibilidade pontual de um serviço seja comunicada de forma clara, sem comprometer a experiência do utilizador nas restantes funcionalidades.

Contingência para indisponibilidade do Keycloak. Toda a autenticação do sistema depende de uma única instância do Keycloak; se esta ficar indisponível, nenhum serviço protegido consegue validar tokens JWT, mesmo que o token já tenha sido emitido anteriormente. Uma evolução futura poderia explorar a colocação em cache das chaves públicas de validação já obtidas, com um período de tolerância, ou a configuração de uma instância secundária do Keycloak em alta disponibilidade, reduzindo o impacto de uma falha do serviço de autenticação.

Gateway como *load balancer*. O gateway Nginx assume atualmente apenas o papel de encaminhamento de pedidos por caminho de URL para uma única instância de cada serviço. Para suportar maior carga, o gateway poderia ser reconfigurado para distribuir pedidos entre múltiplas réplicas de cada serviço, permitindo escalabilidade horizontal real e tolerância a falhas de instâncias individuais, sem alterações à lógica dos serviços de backend.

7.4 Distribuição da Aplicação Móvel

Publicação nas lojas oficiais de aplicações. A aplicação móvel foi desenvolvida com Expo e testada em ambiente de desenvolvimento, mas ainda não foi submetida às lojas oficiais. A publicação na Google Play Store e na Apple App Store é um passo natural para tornar o ShopISEL acessível ao público em geral, exigindo a preparação de processos de *build* de produção, conformidade com as políticas de cada loja e a gestão de atualizações através dos respetivos canais.

Capítulo 8

Conclusão

Este capítulo apresenta uma síntese do trabalho desenvolvido no âmbito do projeto ShopISEL e reflete sobre os principais desafios encontrados. As direções de evolução futura identificadas mas não exploradas no âmbito deste projeto, por restrições de tempo, são apresentadas em detalhe no Capítulo 7.

8.1 Síntese do Trabalho Desenvolvido

O ShopISEL partiu de uma necessidade concreta dos consumidores: organizar compras num contexto de múltiplas insígnias, preços variáveis e informação dispersa. A solução desenvolvida reúne uma aplicação móvel, uma aplicação web e uma infraestrutura backend orientada a serviços, com base de dados PostgreSQL partilhada.

Em termos funcionais, a plataforma já cobre os fluxos principais do projeto: gestão de listas, consulta de preços, alertas de descida de preço, leitura de códigos de barras e apresentação de preços com base na localização do utilizador quando aplicável. Estes elementos mostram que o sistema não ficou apenas pela ideia inicial, mas evoluiu para um conjunto de funcionalidades utilizáveis em cenários concretos de compra.

Do ponto de vista técnico, o maior desafio não foi apenas implementar ecrãs ou endpoints, mas ligar informação proveniente de fontes com estruturas, formatos e níveis de detalhe distintos, com autenticação centralizada, notificação push e sincronização entre serviços sem perder simplicidade de utilização.

Os principais componentes desenvolvidos foram:

- Cinco serviços de backend em ASP.NET Core (listas, produtos, preços, lojas, contas), cada um com persistência relacional em PostgreSQL;
- Um serviço ASP.NET Core com modo *worker* (MatchQueue), executado pelo GitHub Actions, que faz a ponte entre dados de scraping armazenados em MongoDB e a base de dados relacional normalizada;
- Dois scrapers baseados em Playwright para recolha automática de dados do Continente e do Pingo Doce;

- Um orchestrator de scrapers baseado em GitHub Actions reusable workflows;
- Um worker de notificações que integra a base de dados com o Firebase Cloud Messaging;
- Uma aplicação web em React com TypeScript e Vite, com suporte a concorrência otimista nas listas de compras e arquitetura white-label dinâmica injetada em tempo de *build*;
- Uma aplicação móvel em React Native com Expo, com suporte a leitura de código de barras, localização, notificações push e internacionalização;
- Um gateway Nginx como ponto de entrada unificado para as aplicações cliente;
- Pipelines de CI/CD com GitHub Actions para todos os serviços, com publicação automática no GitHub Container Registry e deployment na plataforma Fly.io;
- Mecanismos de concorrência otimista e tratamento centralizado de erros no List Service, com adaptação correspondente nas aplicações cliente;
- Pipeline de personalização white label na aplicação web, activável por parâmetros no workflow de CI/CD sem alteração ao código-fonte partilhado.

8.2 Desafios Encontrados

O desenvolvimento do ShopISEL apresentou desafios técnicos relevantes em várias frentes.

Recolha e normalização de dados. A extração de dados dos websites do Continente e do Pingo Doce revelou-se um processo frágil, dado que as estruturas HTML das páginas podem mudar sem aviso. O uso do Playwright permitiu lidar com páginas de renderização dinâmica, mas a manutenção dos scrapers exige atenção contínua às alterações dos websites fontes. A normalização dos dados provenientes de fontes heterogêneas, com formatos e nomenclaturas distintos, exigiu a criação de uma camada de matching (MatchQueue) que identificasse o mesmo produto em diferentes insígnias.

Isolamento de autenticação em testes. A integração com o Keycloak é essencial para a segurança do sistema, mas representa um ponto de dependência externa nos testes. A solução encontrada, com a criação de um `TestAuthHandler` que simula a autenticação em contexto de teste, permite executar testes de integração completos sem necessidade de uma instância Keycloak externa.

Arquitetura multi-repositório. A gestão de múltiplos repositórios com ciclos de desenvolvimento independentes requer disciplina no versionamento, na comunicação de contratos entre serviços e na coordenação de alterações que afetam múltiplos componentes. A utilização de GitHub Actions e de ficheiros de configuração declarativos (`fly.toml`) contribuiu para mitigar esta complexidade.

Sincronização multi-cliente. A possibilidade de editar a mesma lista a partir da aplicação web e da aplicação móvel impôs a adoção de concorrência otimista no List Service e a adaptação das aplicações cliente ao novo contrato de versão. Este desafio transversal exigiu alterações coordenadas entre backend e frontend, bem como testes que validassem não só o comportamento isolado da API, mas também a experiência do utilizador perante conflitos de

atualização.

Localização e contexto de compra. A funcionalidade de apresentação de preços próximos depende da localização do dispositivo, o que acrescenta valor à decisão do utilizador sem exigir um modelo geográfico pesado no servidor. O sistema pede permissão quando necessário e usa essa informação apenas para ordenar e contextualizar resultados.

8.3 Considerações Finais

O projeto ShopISEL demonstra a viabilidade de uma plataforma digital funcional e coerente, mas também evidencia limites concretos do processo seguido. Contribuíram para essa viabilidade a decomposição por serviços, contratos explícitos entre componentes, contentorização, CI/CD, testes de integração e revisão iterativa das funcionalidades, o que ajudou a manter o sistema evolutivo e verificável.

Ainda assim, o desenvolvimento não correspondeu a um processo A-SDLC completo, tal como enquadrado por Bhati [2]. Não houve delegação autónoma de tarefas de ponta a ponta a agentes, nem um sistema formal de governação com registo sistemático de decisões, validação de cada passo por agentes especializados e controlo contínuo do contexto. O uso de LLMs foi sobretudo assistivo, apoiando implementação, refinamento e validação pontual.

Esta distinção é importante: o valor do projeto está menos na adesão total a um paradigma agentic e mais na combinação pragmática entre desenvolvimento orientado por humanos, automação de entrega e apoio pontual por IA. Em comparação com uma abordagem A-SDLC mais madura, o trabalho ainda fica aquém de práticas como orquestração autónoma de tarefas, verificação sistemática multiagente e rastreabilidade completa das decisões assistidas por IA.

Em suma, o ShopISEL representa um primeiro passo sólido para uma plataforma de apoio à decisão de compra que, com as direções de evolução apresentadas no Capítulo 7, tem potencial para se tornar uma ferramenta genuinamente útil no quotidiano dos consumidores portugueses.

Bibliografia

- [1] AnyList, Inc. (2026). Anylist: Grocery shopping list. <https://www.anylist.com>. Accessed: 2026-05-10.
- [2] Bhati, H. (2026). Agentic ai in the software development lifecycle: Architecture, empirical evidence, and the reshaping of software engineering. <https://arxiv.org/abs/2604.26275>. Accessed: 2026-07-06.
- [3] Bring! Labs AG (2026). Bring! shopping list. <https://www.getbring.com>. Accessed: 2026-05-10.
- [4] Docker (2026). What is docker? <https://docs.docker.com/get-started/docker-overview/>. Accessed: 2026-04-27.
- [5] Expo (2025). Introduction to expo. <https://docs.expo.dev/get-started/introduction/>. Accessed: 2026-04-27.
- [6] Flipp Corp (2026). Flipp – weekly ads & coupons. <https://flipp.com>. Accessed: 2026-05-10.
- [7] GitHub (2026). Github actions documentation. <https://docs.github.com/en/actions/>. Accessed: 2026-04-27.
- [8] Google (2026). Firebase cloud messaging. <https://firebase.google.com/docs/cloud-messaging>. Accessed: 2026-05-10.
- [9] GS1 (2026). Ean/upc barcodes. <https://www.gs1.org/standards/barcodes/ean-upc>. Accessed: 2026-06-01.
- [10] Headcode Ltd (2026). Ourgroceries shopping lists. <https://www.ourgroceries.com>. Accessed: 2026-05-10.
- [11] Jerónimo Martins (2026). Pingo doce – aplicação mobile. <https://www.pingodoce.pt/aplicacao>. Accessed: 2026-05-10.
- [12] Keycloak (2026). Securing applications and services with openid connect. <https://www.keycloak.org/securing-apps/oidc-layers>. Accessed: 2026-04-27.
- [13] Kuantokusta (2026). Kuantokusta – comparador de preços. <https://www.kuantokusta.pt/public/quem-somos>. Accessed: 2026-06-01.

- [14] Kung, H. T. and Robinson, J. T. (1981). On optimistic methods for concurrency control. *ACM Transactions on Database Systems*, 6(2):213–226.
- [15] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions and reversals. *Doklady Akademii Nauk SSSR*, 163(4):845–848. Accessed: 2026-06-01.
- [16] Mark Lawlor (2026). Nativewind documentation. <https://www.nativewind.dev>. Accessed: 2026-05-10.
- [17] Meta Open Source (2026). Built-in react hooks. <https://react.dev/reference/react/hooks>. Accessed: 2026-04-27.
- [18] Microsoft (2025). Apis overview: Minimal apis. <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/minimal-apis/overview>. Accessed: 2026-04-27.
- [19] Microsoft (2026a). Integration tests in asp.net core. <https://learn.microsoft.com/en-us/aspnet/core/test/integration-tests>. Accessed: 2026-05-10.
- [20] Microsoft (2026b). Playwright documentation. <https://playwright.dev/docs/intro>. Accessed: 2026-05-10.
- [21] MongoDB, Inc. (2026). Mongodb documentation. <https://www.mongodb.com/docs/>. Accessed: 2026-05-10.
- [22] Npgsql (2026). Npgsql entity framework core provider. <https://www.npgsql.org/efcore/>. Accessed: 2026-04-27.
- [23] Open Food Facts (2026). Open food facts api documentation. <https://openfoodfacts.github.io/openfoodfacts-server/api/>. Accessed: 2026-06-01.
- [24] Pricena (2026). Pricena – price comparison. <https://www.pricena.com>. Accessed: 2026-05-10.
- [25] Sonae MC (2026). Continente – aplicação mobile. <https://www.continente.pt/app>. Accessed: 2026-05-10.
- [26] The PostgreSQL Global Development Group (2026). Postgresql documentation. <https://www.postgresql.org/docs/>. Accessed: 2026-04-27.
- [27] xUnit.net (2026). xunit.net documentation. <https://xunit.net>. Accessed: 2026-05-10.
- [28] Zanevych, O. (2024). Advancing web development: A comparative analysis of modern frameworks for rest and graphql back-end services. *Grail of Science*, (37):216–228.